НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«Розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконала студентка групи АІ-244
Гермікова Д.А.
Перевірив: ас. Баранов О.М.

Одеса – 2026

Мета: Запустити мовну модель локально й керувати нею з коду, розуміти, що локальна модель і хмарна – це один і той самий інтерфейс і вміти перемикатися між ними зміною двох рядків, вимірювати продуктивність інференсу й пояснювати, з чого складається затримка, емпірично показати вплив temperature і top_p на стабільність відповідей.

Підготовка середовища для виконання завдання 1:

Крок 1. Встановити Ollama і завантажити моделі.

За посиланням http://ollama.com, ми встановили Ollama та відкривши його підключили до GitHub аккаунту. Відкривши термінал, перевірили що дійсно все підключилося (Рис.1.1).

```
Last login: Wed Sep 30 15:20:17 on ttys000
deniza@Noutbuk-Deniza ~ % ollama --version
ollama version is 0.35.0
deniza@Noutbuk-Deniza ~ %
```

Рис.1.1 – Перевірка завантаження Ollama у терміналі.

Завантажимо моделі (Рис.1.2).

```
deniza@Noutbuk-Deniza ~ % ollama pull qwen3:4b
pulling manifest
pulling 3e4cb1417446: 100% ▕▕▕
```

```
deniza@Noutbuk-Deniza ~ % ollama list
NAME            ID              SIZE      MODIFIED
llama3.2:3b     a80c4f17acd5    2.0 GB    8 minutes ago
qwen3:4b        359d7dd4bcda    2.5 GB    12 minutes ago
deniza@Noutbuk-Deniza ~ %
```

Рис.1.3 – Список моделей та їхній розмір на диску.

Виконаємо запит до моделі, щоб переконатися що модель "жива" (Рис.1.4).

```
deniza@Noutbuk-Deniza ~ % ollama run qwen3:4b "Надайте інформацію про вашу модель"
Thinking...
Хорошо, мне нужно предоставить информацию о модели Qwen. Я должен убедиться, что я правильно понимаю, что
пользователь хочет. Он спрашивает о моей модели, то есть, вероятно, хочет знать, какая модель я
использую, мои способности, какие у меня функции и т.д.

Сначала я должен вспомнить, что я Qwen, который является сверхмасштабной языковой моделью, разработанной
Alibaba Cloud. Нужно уточнить, что я являюсь последней версией Qwen3, которая обладает высокой
производительностью и широкими возможностями.

Далее, нужно описать основные функции и способности модели: например, поддержка нескольких языков,
способность обрабатывать разные задачи (текст генерация, ответы на вопросы, логические рассуждения,
программирование и т.д.), а также возможность генерации кода.

Также важно упомянуть, что модель обучена на больших данных и имеет обширный словарный запас, что
позволяет ей понимать и генерировать текст на высоком уровне.

Еще нужно проверить, нет ли ошибок в терминах. Например, правильно ли называется модель Qwen3, и есть ли
у нее другие названия или версии. В предыдущих ответах я использовал Qwen3, так что, наверное, это верно.

Нужно структурировать ответ так, чтобы он был понятен и содержал все важные моменты. Возможно, стоит
начать с того, что я Qwen3, потом перечислить способности и функции.

Проверю, нет ли ошибок в описании: например, модель обучена на данных до 2024 года, поддерживает 100+
языков, может генерировать код на разных языках программирования. Важно не перегружать ответ деталями, но
дать краткую и полезную информацию.

Также стоит уточнить, что модель разработана Alibaba Cloud и используется для различных применений.
Возможно, пользователь хочет знать, какую роль я играю в системе, но в данном случае он спрашивает именно
про модель.

Нужно избегать технических терминов, чтобы ответ был понятен широкой аудитории. Проверю, что все
правильно.

Допустим, такой ответ будет правильным: "Я являюсь сверхмасштабной языковой моделью Qwen3, разработанной
Alibaba Cloud. Моя модель обладает высокой производительностью и поддерживает множество задач, включая
генерацию текста, ответы на вопросы, логические рассуждения и программирование. Я поддерживаю более 100
языков и могу генерировать код на различных языках программирования. Моя модель обучена на обширных
данных и способна обеспечивать высококачественные результаты для различных задач."

Но нужно проверить, нет ли ошибок. Например, в некоторых источниках может быть другое название модели, но
в контексте Qwen3 это верно.

Также важно уточнить, что это последняя версия, возможно, Qwen3 — это самая новая версия. В предыдущих
ответах я использовал Qwen3, так что, наверное, это правильно.

Хорошо, составлю ответ на русском языке, как просил пользователь (он написал на русском: "Надайте
інформацію про вашу модель"). Значит, нужно ответить на русском.

Проверю, как правильно перевести "Qwen3" на русский. Обычно модели сохраняют название на английском, но
можно уточнить. В русском языке обычно пишут как "Qwen3" или "модель Qwen3".

Также нужно убедиться, что структура ответа правильна и соответствует запросу.

Итак, составлю ответ:

"Я являюсь сверхмасштабной языковой моделью Qwen3, разработанной Alibaba Cloud. Моя модель обладает
высокой производительностью и поддерживает множество задач, включая генерацию текста, ответы на вопросы,
логические рассуждения и программирование. Я поддерживаю более 1.000 языков и могу генерировать код на
различных языках программирования. Моя модель обучена на обширных данных и способна обеспечивать
высококачественные результаты для различных задач."

Подожду, в предыдущих ответах, возможно, я ошибся с количеством языков. В документации Qwen3 указано, что
поддержка более 100 языков, а не 1000. Нужно исправить.

Также, в русском языке лучше использовать "более 100 языков", а не "1.000".

Исправлю: "Я поддерживаю более 100 языков".
```

```
Также, может быть, стоит уточнить, что модель обучена на данных до 2024 года, но если это не указано в
официальной документации, лучше не добавлять.

Проверю еще раз структуру.

Возможно, ответ будет таким:

"Я являюсь сверхмасштабной языковой моделью Qwen3, разработанной Alibaba Cloud. Моя модель обладает
высокой производительностью и поддерживает множество задач, включая генерацию текста, ответы на вопросы,
логические рассуждения и программирование. Я поддерживаю более 100 языков и могу генерировать код на
различных языках программирования. Моя модель обучена на обширных данных и способна обеспечивать
высококачественные результаты для различных задач."

Да, это выглядит правильным.

Также важно, чтобы ответ был кратким и не содержал лишней информации, чтобы пользователь получил нужную
информацию без перегрузки.

Проверю, нет ли ошибок в грамматике на русском.

"Я являюсь" — правильно.

"сверхмасштабной языковой моделью" — возможно, лучше "сверхмасштабной языковой моделью" или "моделью
сверхмасштабной языковой модели", но в контексте это правильно.

"разработанной Alibaba Cloud" — правильно.

"Моя модель обладает высокой производительностью и поддерживает множество задач" — правильно.

"включая генерацию текста, ответы на вопросы, логические рассуждения и программирование" — правильно.

"Я поддерживаю более 100 языков" — правильно.

"и могу генерировать код на различных языках программирования" — правильно.

"Моя модель обучена на обширных данных и способна обеспечивать высококачественные результаты для
различных задач" — правильно.

Вроде все правильно. Думаю, такой ответ подойдет.
...done thinking.

Я являюсь сверхмасштабной языковой моделью Qwen3, разработанной Alibaba Cloud. Моя модель обладает
высокой производительностью и поддерживает множество задач, включая генерацию текста, ответы на вопросы,
логические рассуждения и программирование. Я поддерживаю более 100 языков и могу генерировать код на
различных языках программирования. Моя модель обучена на обширных данных и способна обеспечивать
высококачественные результаты для различных задач.
```

Рис.1.4 – Виконаний запит до моделі.

Крок 2. Побачити, що модель – це звичайний веб-сервер.

Виконаємо в терміналі запит напряму з використанням моделі qwen3:4b (Рис.1.5).

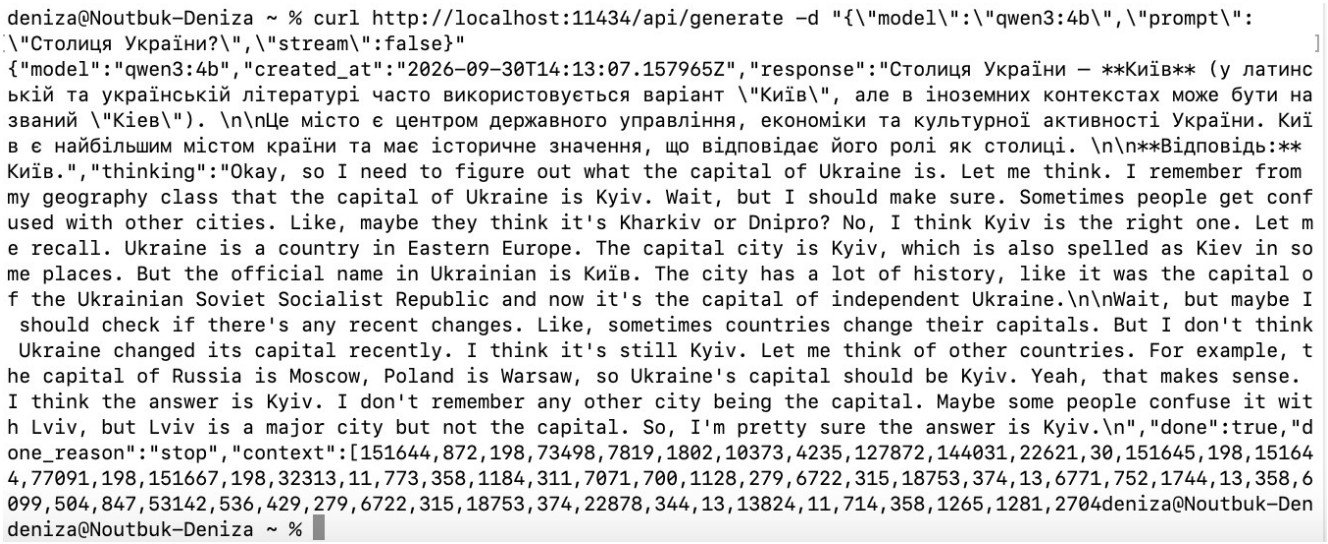


Рис.1.5 – Виконання запиту «Столиця України?» до моделі qwen3:4b через API Ollama та отримання відповіді.

Виконання завдання:

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі).

Крок 1. Просте фактичне питання. Виконати для обох моделей.

Для порівняння міркувальної моделі qwen3:4b та інструктивної моделі llama3.2:3b виконано однаковий запит через ендпойнт /api/generate:
«Столиця України? Відповідай одним словом.»

Обидві моделі надали правильну відповідь — Київ. У відповіді qwen3:4b додатково присутнє поле thinking, що містить внутрішній процес міркування, тоді як llama3.2:3b одразу повернула кінцеву відповідь (Рис.1.6).

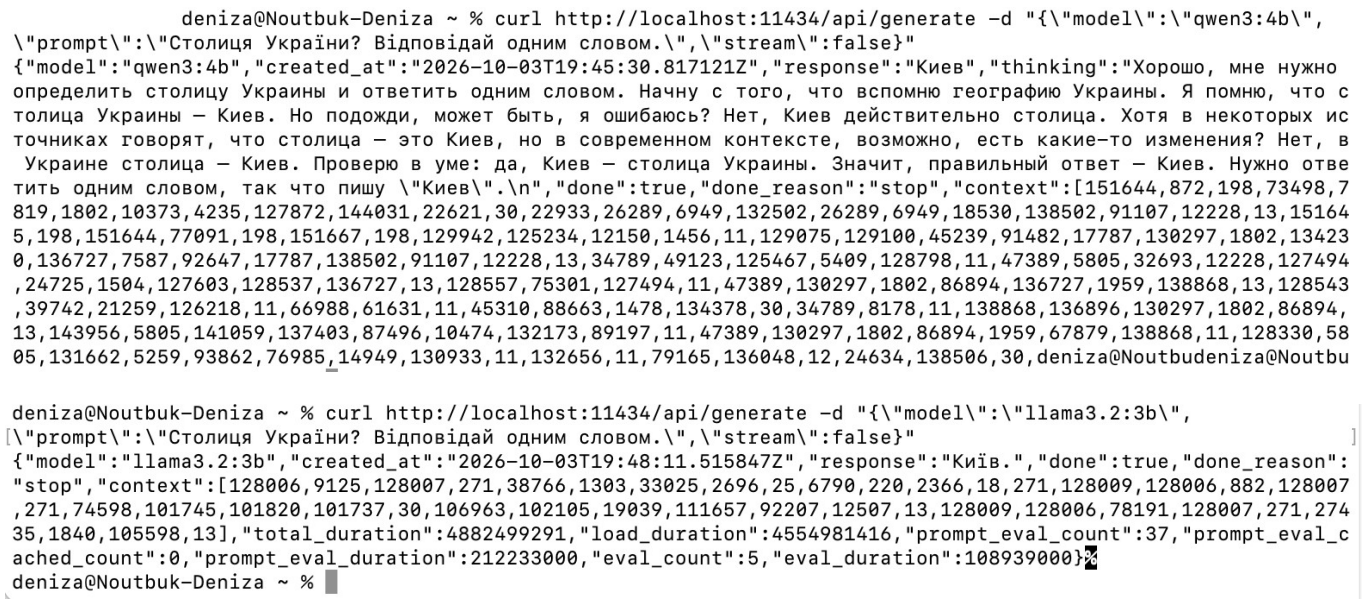


Рис.1.6 – Результати виконання однакового фактичного запиту для моделей qwen3:4b та llama3.2:3b через API Ollama.

Крок 2. Задача на міркування. Виконати для обох моделей по 5 разів.

Після цього кожну модель п’ять разів перевірили на задачі, що потребує логічного міркування, щоб порівняти правильність відповідей, час виконання та кількість згенерованих токенів (Рис.1.7) та (Рис.1.8).



Рис.1.7 – Результат запуску задачі на міркування для моделі qwen3:4b.

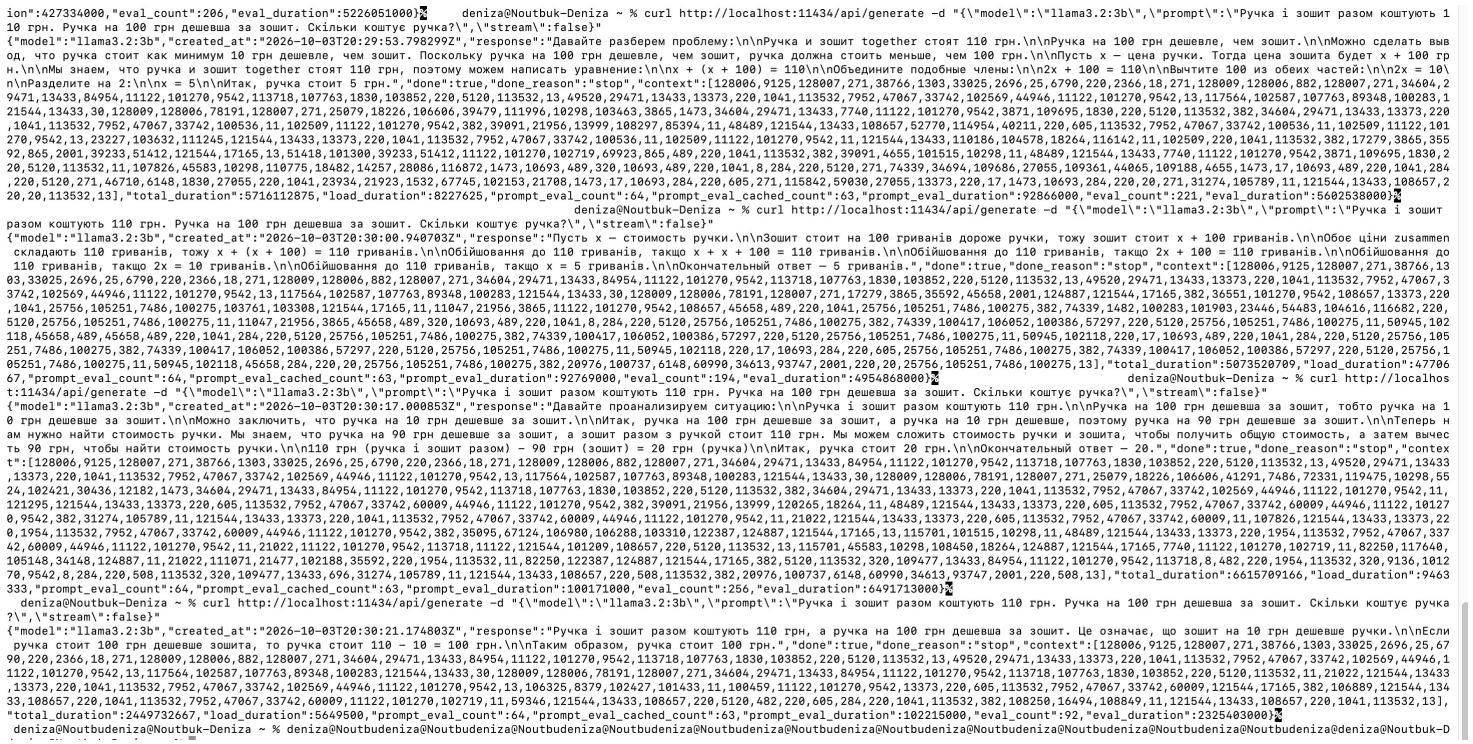


Рис.1.8 – Результат запуску задачі на міркування для моделі llama3.2:3b.

Під час виконання задачі на міркування модель qwen3:4b правильно відповіла у 5 із 5 запусків, тоді як llama3.2:3b дала правильну відповідь у 2 із 5 запусків. Водночас llama3.2:3b працювала значно швидше за qwen3:4b, що показує компроміс між якістю міркування та швидкістю генерації. Отже, міркувальна модель краще впоралася із задачею, що потребує логічного аналізу, але потребувала більше часу та токенів для формування відповіді.

Заповнимо за результатами тестів (Таблиця 1) та (Таблиця 2).

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	316	5
Час, с	20,57	4,88
Токенів/с	26,24	45,90

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	2 / 5
eval_count	1266,8	193,8
Час, с	49,76	5,96

Неправильні відповіді:
- 105 грн Модель, ймовірно, неправильно інтерпретувала умову про різницю в 100 грн і не врахувала, що ручка має бути дешевшою за зошит на 100 грн.
- 20 грн Модель не виконала логічного міркування до кінця і дала число, яке не задовольняє умову завдання.
- 100 грн Тут модель фактично обрала величину різниці як вартість ручки, хоча за умовою 100 грн – це не ціна ручки, а різниця між ціною ручки та зошита.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальної моделі?
Відповідь: Додаткові токени були використані для внутрішнього міркування, перевірки проміжних кроків та формування остаточної відповіді. Це добре видно за великим значенням eval_count у моделі qwen3:4b.
2. На яких типах задач міркувальні моделі дають перевагу?
Відповідь: Міркувальні моделі мають перевагу в задачах, де потрібно виконувати послідовні логічні кроки, обчислення, аналіз або перевірку результату. Для простих фактичних запитань їх використання часто є недоцільним через більші витрати часу та токенів.
3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?
Відповідь: Міркувальні моделі додатково оптимізуються для виконання складних багатокрокових міркувань і розв'язання задач із проміжними кроками. Інструктивні моделі більше орієнтовані на безпосереднє виконання інструкції та швидке формування відповіді.
4. Чи означає більша кількість токенів, що модель «розумніша»?
Відповідь: Ні. Велика кількість токенів означає, що модель витратила більше обчислювального ресурсу на формування відповіді, але сама по собі не є показником її інтелекту або якості.
5. Як пояснити різницю у швидкості генерації між моделями?
Відповідь: Швидкість генерації залежить від розміру моделі, її архітектури та обсягу обчислень, необхідних для створення відповіді. У проведеному експерименті llama3.2:3b генерувала токени швидше за qwen3:4b, а qwen3:4b витрачала значно більше часу на міркування.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 4,22 разів. На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе 398,1 с проти 47,7 с, що робить міркувальну модель непридатною для налагодження.

Тож, вибір моделі повинен залежати від характеру поставленої задачі. Для простих запитів доцільніше використовувати інструктивну модель, яка швидко формує відповідь, тоді як для задач, що потребують логічного аналізу, міркувальна модель може забезпечити вищу точність ціною більшого часу виконання та кількості токенів.

Загальні висновки

1. Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;
2. Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;
3. Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;
4. Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;
5. Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.

Підготовка середовища для виконання завдання 2:

Крок 3. Керувати моделлю з програми на Python

1. Встановіть Python. Спочатку перевірте, чи встановлений Python

Перевірено Python встановлений (Рис.2.1).

```
deniza@Noutbuk-Deniza ~ % python3 --version
Python 3.9.6
```

Рис.2.1 – Перевірка встановлення Python.

2. Створіть папку проєкту та віртуальне середовище:
mkdir lab1 && cd lab1
python -m venv venv
Активуйте його:
venv\Scripts\activate.bat
Ознака успіху – на початку рядка з’явиться (venv) (Рис.2.2).

```
deniza@Noutbuk-Deniza lab1 % source venv/bin/activate
(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.2.2 –Успішне створення віртуального середовища та папки проєкту лаб1.

3. Встановіть бібліотеку openai :
OpenAI-сумісний API —- компанія OpenAI свого часу задала формат таких запитів, і він став галузевим стандартом. Тепер майже всі — і Ollama, і Google, і Groq — приймають запити в тому самому форматі. Практичний наслідок: студент пише код один раз, а потім перемикається між локальною моделлю і хмарною, змінивши одну адресу. Встановимо цю бібліотеку (Рис.2.3).

```
(venv) deniza@Noutbuk-Deniza lab1 % pip install openai python-dotenv
Collecting openai
  Downloading openai-2.48.0-py3-none-any.whl (1.6 MB)
     |████████████████████████████████| 1.6 MB 4.2 MB/s
Collecting python-dotenv
  Downloading python_dotenv-1.2.1-py3-none-any.whl (21 kB)
Collecting jiter<1,>=0.10.0
  Downloading jiter-0.16.0-cp39-cp39-macosx_11_0_arm64.whl (315 kB)
     |████████████████████████████████| 315 kB 48.8 MB/s
Collecting sniffio
  Downloading sniffio-1.3.1-py3-none-any.whl (10 kB)
Collecting httpx<1,>=0.23.0
  Downloading httpx-0.28.1-py3-none-any.whl (73 kB)
     |████████████████████████████████| 73 kB 14.1 MB/s
Collecting tqdm>4
  Downloading tqdm-4.70.1-py3-none-any.whl (80 kB)
     |████████████████████████████████| 80 kB 49.3 MB/s
Collecting distro<2,>=1.7.0
  Downloading distro-1.9.0-py3-none-any.whl (20 kB)
Collecting pydantic<3,>=1.9.0
  Downloading pydantic-2.13.5-py3-none-any.whl (472 kB)
     |████████████████████████████████| 472 kB 61.7 MB/s
Collecting anyio<5,>=3.5.0
  Downloading anyio-4.12.1-py3-none-any.whl (113 kB)
     |████████████████████████████████| 113 kB 45.8 MB/s
Collecting typing-extensions<5,>=4.14
  Using cached typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Collecting exceptiongroup>=1.0.2
  Using cached exceptiongroup-1.3.1-py3-none-any.whl (16 kB)
Collecting idna>=2.8
  Downloading idna-3.20-py3-none-any.whl (69 kB)
     |████████████████████████████████| 69 kB 37.1 MB/s
Collecting certifi
  Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
     |████████████████████████████████| 136 kB 34.7 MB/s
Collecting httpcore==1.*
  Downloading httpcore-1.0.9-py3-none-any.whl (78 kB)
     |████████████████████████████████| 78 kB 51.0 MB/s
Collecting h11>=0.16
  Downloading h11-0.16.0-py3-none-any.whl (37 kB)
Collecting annotated-types>=0.6.0
  Downloading annotated_types-0.7.0-py3-none-any.whl (13 kB)
Collecting typing-inspection>=0.4.2
  Downloading typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Collecting pydantic-core==2.46.5
  Downloading pydantic_core-2.46.5-cp39-cp39-macosx_11_0_arm64.whl (1.9 MB)
     |████████████████████████████████| 1.9 MB 67.7 MB/s
Installing collected packages: typing-extensions, idna, h11, exceptiongroup, certifi, typing-inspection, pyda
ntic-core, httpcore, anyio, annotated-types, tqdm, sniffio, pydantic, jiter, httpx, distro, python-dotenv, op
enai
Successfully installed annotated-types-0.7.0 anyio-4.12.1 certifi-2026.7.22 distro-1.9.0 exceptiongroup-1.3.1
 h11-0.16.0 httpcore-1.0.9 httpx-0.28.1 idna-3.20 jiter-0.16.0 openai-2.48.0 pydantic-2.13.5 pydantic-core-2.
46.5 python-dotenv-1.2.1 sniffio-1.3.1 tqdm-4.70.1 typing-extensions-4.16.0 typing-inspection-0.4.2
WARNING: You are using pip version 21.2.4; however, version 26.0.1 is available.
You should consider upgrading via the '/Users/deniza/lab1/venv/bin/python3 -m pip install --upgrade pip' comm
and.
(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.2.3 –Успішне встановлення бібліотеки openai.

4. Створіть файл hello.py. Створюємо файл hello.py (Рис.2.4).

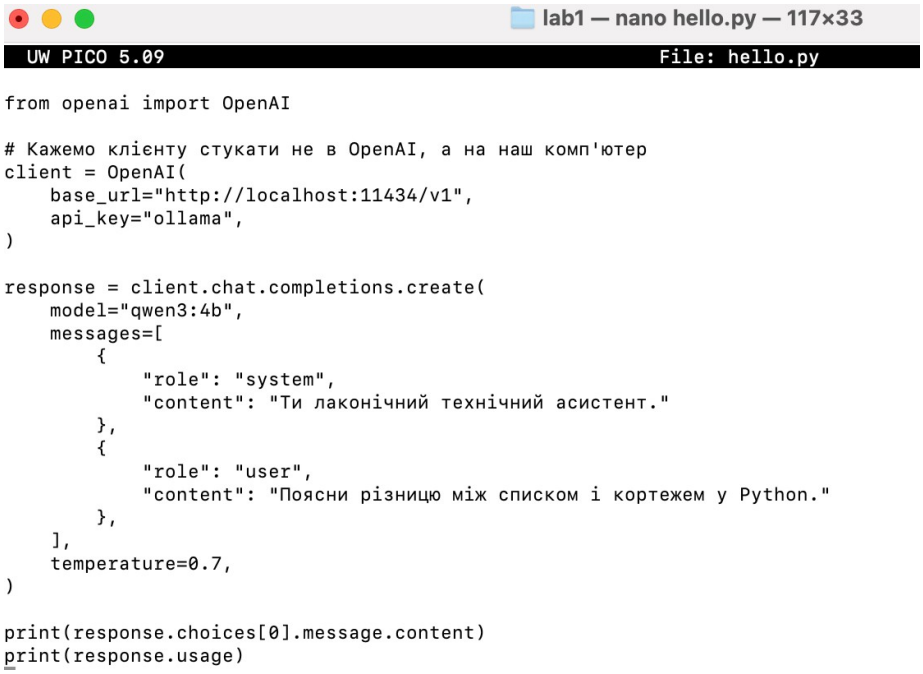


Рис.2.4 – Створення файлу hello.py.

Запуск програми (Рис.2.5).

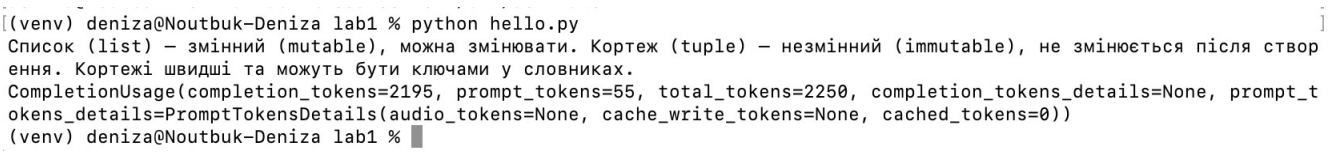


Рис.2.5 – Запуск програми hello.py.

Виконання завдання:

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Маємо варіант номер 2 (за журналом 22). Тож виконаємо варіант: 2. Поясни різницю між стеком і чергою.

Крок 1. Створити файл hello.py з нашим варіантом.

Змінемо файл hello.py відповідно до свого варіанта та задамо system-повідомлення «Ти лаконічний технічний асистент.» (Рис.2.6).



Рис.2.6 – Код програми hello.py для моделі llama3.2:3b з першим system-повідомленням.

Крок 2. Запустити: python hello.py . Зберегти відповідь.

Виконаємо програму hello.py для моделі llama3.2:3b та отримано відповідь на питання про різницю між стеком і чергою. Результат виконання (Рис.2.7).

```
(venv) deniza@Noutbuk-Deniza lab1 % python hello.py
Стек і черга — дві класичні терміни з гри, які часто використовуються в різному контекстах. Давайте розгледим різн
ісю між ними:

**Стек:**
Стек — це гра, у якому гравці використовують карту, змагатися за виграш. Карту подрібнюють в стак, а гравці намага
лися відпадтися з стаков, щоб виграти карту. Стек часто грається з картами, зазвичай 52 картами з п'ятьма цифрами
(Ace, Дамка, Крейг, Риба, Буба) та чотирьох королями (король, королева, буба, риба).

**Черга:**
Черга — це гра з козликом, який використовує ладью для перемагання гравців. У черги беруть участь п'ять ладьй з п'
ятьма цифрами (Ace, Дамка, Крейг, Риба, Буба) та п'ять козлів. Гравці намагалися виграти ладью за допомогою ладьї,
 змагатися за виграш. Черга часто грається з ладьями, зазвичай п'ять ладьй з п'ятьма цифрами.

Різниця між стеком та чергою полягає в тому, що:

* Стек грається з картами, а черга грається з ладьями.
* Стек не використовує козлів, а черга використовує козлів для перемагування гравців.
* Стек часто грається з картами з цифр, а черга грається з ладьями з цифрами.

Тепер, коли ви знаєте різнісю між стеком та чергою, можете грати з ними з цьомістю!
CompletionUsage(completion_tokens=447, prompt_tokens=51, total_tokens=498, completion_tokens_details=None, prompt_
tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=0))
(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.2.7 – Результат виконання програми для моделі llama3.2:3b за варіантом 2.

Крок 3. Змінити system-повідомлення на:W.

System-повідомлення змінено на «Відповідай лише українською, максимум двома реченнями.» та повторно запущено програму (Рис.2.8).

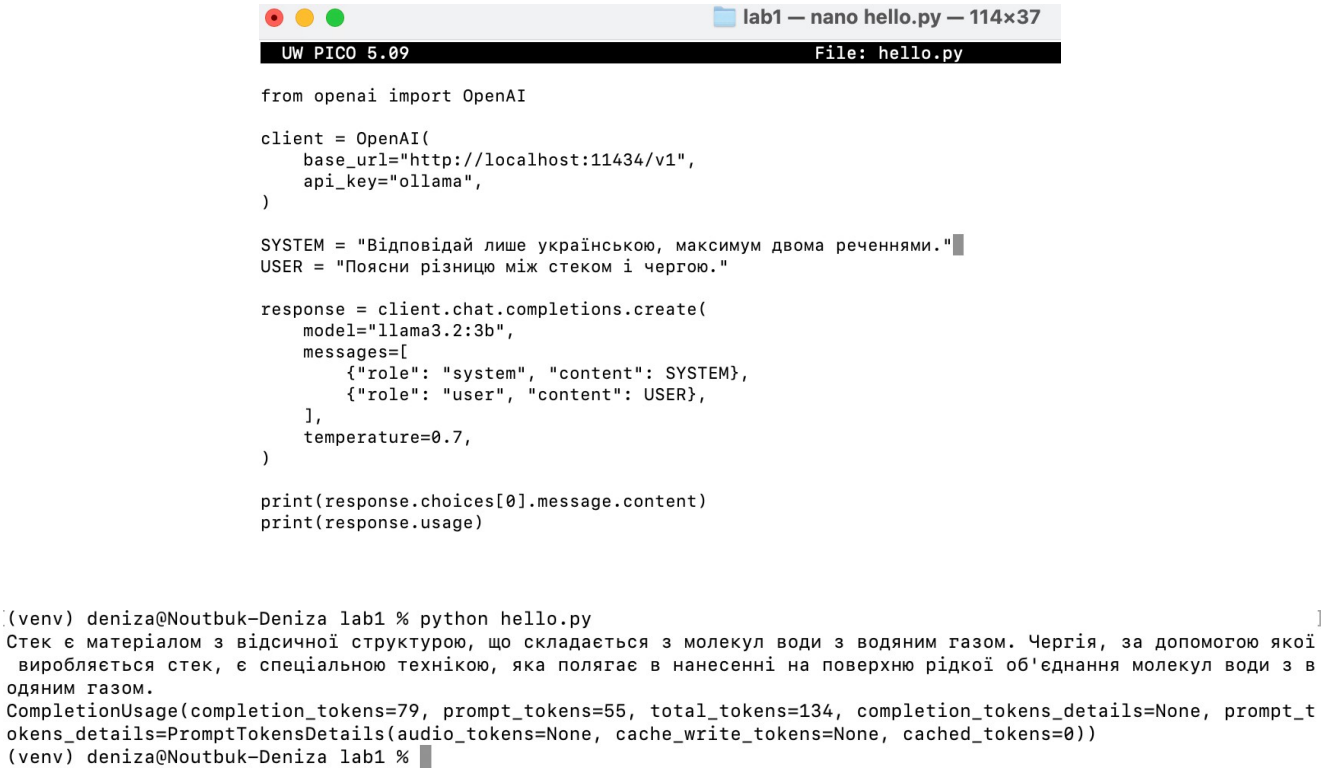
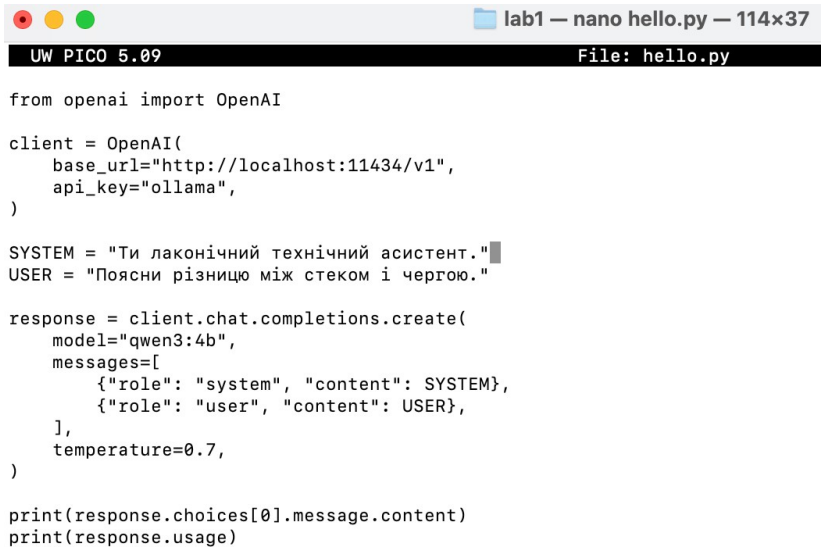


Рис.2.8 – Результат виконання програми llama3.2:3b зі зміненим system-повідомленням.

Крок 4. Повторити кроки 1–3 для моделі qwen3:4b (змінити model у коді).

У програмі змінено модель на qwen3:4b та повторено запуск із двома варіантами system-повідомлення (Рис.2.9–2.10).



```
(venv) deniza@Noutbuk-Deniza lab1 % python hello.py
Стек (LIFO) — останній доданий елемент видаляється першим.
Черга (FIFO) — перший доданий елемент видаляється першим.
Різниця: принцип обробки даних (LIFO vs FIFO).
CompletionUsage(completion_tokens=1907, prompt_tokens=53, total_tokens=1960, completion_tokens_details=None, promp
t_tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=0))
(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.2.9 – Результат виконання програми для моделі qwen3:4b з system-повідомленням «Ти лаконічний технічний асистент.»

```
  UW PICO 5.09                                          File: hello.py

from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:11434/v1",
    api_key="ollama",
)

SYSTEM = "Відповідай лише українською, максимум двома реченнями."
USER = "Поясни різницю між стеком і чергою."

response = client.chat.completions.create(
    model="qwen3:4b",
    messages=[
        {"role": "system", "content": SYSTEM},
        {"role": "user", "content": USER},
    ],
    temperature=0.7,
)

print(response.choices[0].message.content)
print(response.usage)
```

```
(venv) deniza@Noutbuk-Deniza lab1 % python hello.py
Стек (LIFO) дозволяє додавати і видаляти елементи тільки з одного краю, останній введений вийде першим. Черга (FIF
O) працює за принципом першого ввідженого — перший вийде, дозволяючи додавати і видаляти елементи з двох країв.
CompletionUsage(completion_tokens=826, prompt_tokens=62, total_tokens=888, completion_tokens_details=None, prompt_
tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=0))
(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.2.10 – Результат виконання програми для моделі qwen3:4b зі зміненим system-повідомленням.

Крок 5. Заповнимо таблиці (на основі з даними з response.usage).

Таблиця 3 – Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	1020	українська	12	Ні
llama3.2:3b	українською, 2 речення	184	українська	2	Так
qwen3:4b	лаконічний асистент	219	українська	2	Так
qwen3:4b	українською, 2 речення	209	українська	2	Так

Таблиця 4 – Витрати токенів

Модель	System	prompt_tokens	completion_to kens	total_tokens
llama3.2:3b	лаконічний асистент	51	447	498
llama3.2:3b	українською, 2 речення	55	79	134
qwen3:4b	лаконічний асистент	53	1907	1960
qwen3:4b	українською, 2 речення	62	826	888

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?
Відповідь: Точніше дотрималась обмеження модель qwen3:4b, оскільки в обох запусках вона дала відповідь із двох речень. llama3.2:3b виконала це обмеження лише після зміни system-повідомлення.
2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?
Відповідь: Ні, у наведених результатах обидві моделі відповідали українською мовою.
3. Як змінився prompt_tokens при зміні system-повідомлення і чому?
Відповідь: У llama3.2:3b кількість prompt_tokens збільшилася з 51 до 55, а у qwen3:4b — з 53 до 62. Це відбулося тому, що змінене system-повідомлення містить більше тексту і також входить до вхідного запиту моделі.
4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: qwen3:4b є міркувальною моделлю і витрачає додаткові токени на внутрішнє міркування перед формуванням видимої відповіді. Тому completion_tokens у неї може бути значно більшим, навіть якщо кінцевий текст короткий.

Крок 7. Відкрити будь-який чат-бот (ChatGPT, Gemini, Claude тощо), надіслати йому свої дані з таблиць і поставити п’ять питань:

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Різні моделі по-різному навчені виконувати інструкції та керувати форматом відповіді. Міркувальна модель може витрачати більше ресурсів на обробку інструкції, тоді як інструктивна модель може швидше переходити до формування відповіді.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: System-повідомлення задає загальні правила поведінки моделі, а user-повідомлення містить конкретне завдання. Зазвичай system-повідомлення має вищий пріоритет, тому модель повинна намагатися дотримуватися його інструкцій.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Міркувальна модель може використовувати додаткові токени для внутрішнього аналізу та побудови відповіді. Ці витрати не обов'язково відображаються у довжині видимого тексту.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: Якщо інструкції суперечать одна одній, модель зазвичай надає перевагу system-повідомленню, оскільки воно має вищий пріоритет. Однак конкретна поведінка може залежати від моделі та способу її налаштування.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Малі моделі мають менше параметрів і зазвичай гірше справляються зі складними або одночасними обмеженнями. Через це вони можуть частіше порушувати задану мову, формат або довжину відповіді.

Висновок за завданням 2

Отже за результатами можно зробити висновки, що System-повідомлення дозволяє керувати поведінкою моделі, задаючи її роль, стиль і формат відповіді, проте не гарантує повного дотримання інструкцій. Для отримання потрібного формату часто достатньо чіткої інструкції, але іноді модель може її ігнорувати або виконувати не повністю. У такому випадку варто уточнити вимоги, додати приклад бажаної відповіді та перевірити результат.

Підготовка середовища для виконання завдання 3:

Крок 4. Підключити хмарну модель.

Порядок дій:

1. Отримати ключ доступу.

Що таке API-ключ. Це довгий рядок символів, який замінює логін і пароль для програм. Коли ваш код надсилає запит на чужий сервер, він додає цей ключ, і сервер розуміє, чий це запит і чи не вичерпано ліміт. Ключ прирівнюється до пароля: хто його має — витрачає ваш ліміт від вашого імені.
Зареєструйтеся в одного з провайдерів з безкоштовним тарифом (картка не потрібна):

- Google AI Studio — найщедріший безкоштовний ліміт, моделі сімейства Gemini Flash;
- Groq — найшвидша генерація, працює на спеціальному «залізі», моделі з відкритою вагою.

У розділі API keys створіть новий ключ і одразу скопіюйте його — більшість сервісів показують ключ лише один раз. Знайдіть у документації провайдера два значення, вони знадобляться в коді:
base_url — адреса, куди слати запити (наприклад, для Groq це https://api.groq.com/openai/v1); Виконала у Google AI Studio (Рис.3.1).

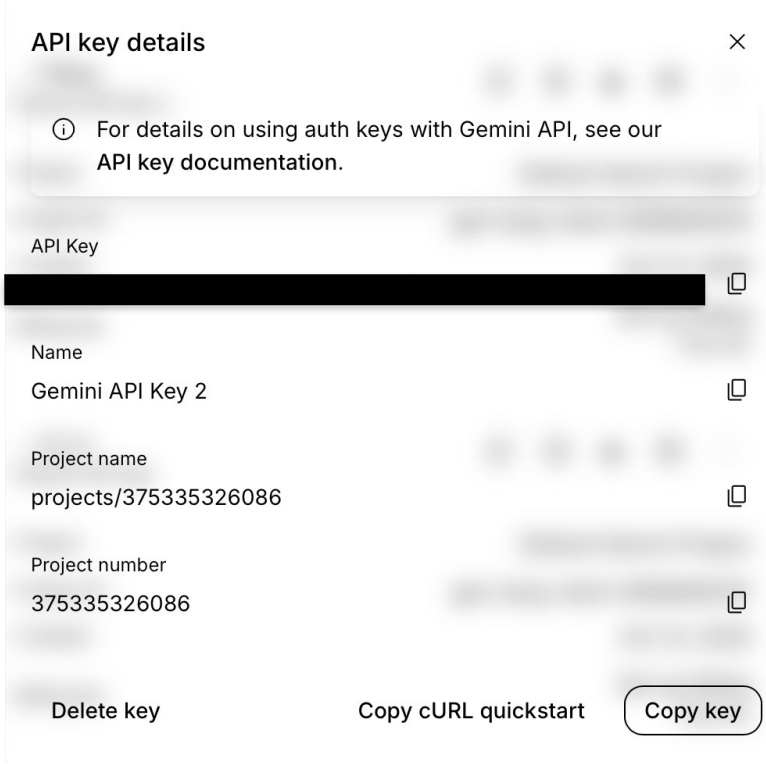


Рис.3.1 – Результат створення ключа.

2. Приховати ключ від чужих очей

У папці проєкту створіть файл з назвою рівно .env (з крапкою на початку):
CLOUD_API_KEY=тут_ваш_ключ_без_лапок
Створіть файл .gitignore і додайте туди рядок:
.env
venv/
.gitignore — це список файлів, які Git ігнорує й ніколи не відправляє на GitHub.

Перевірте, що спрацювало. Виконайте:
git status
Файл .env не має з’явитися у списку. Якщо він там є – ви або створили .gitignore не в тій папці, або встигли додати .env у Git раніше. Все зроблено та працює (Рис.3.2).

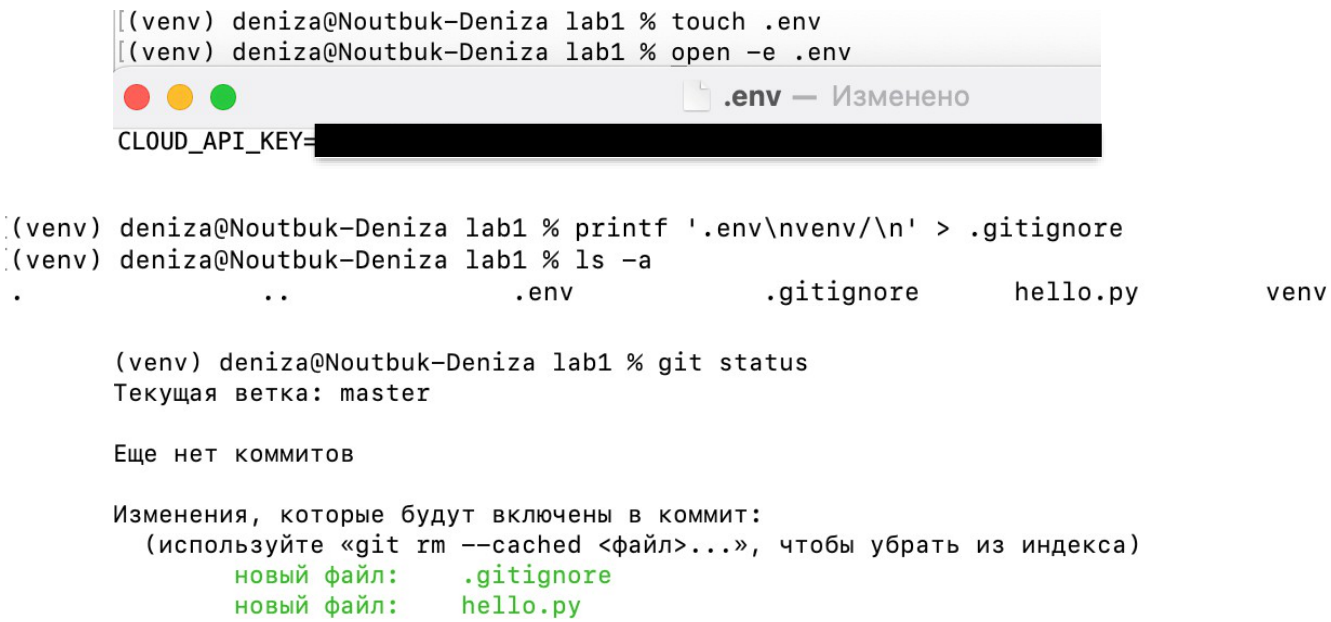


Рис.3.2 – Приховування ключа.

Виконання завдання:

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Маємо варіант номер 2 (за журналом 22). Тож виконаємо варіант: 2. Поясни різницю між стеком і чергою. Коротко.

Крок 1. Отримати ключ, base_url та точну назву моделі.

Для Google Gemini такий base_url: base_url=«https://generativelanguage.googleapis.com/v1beta/openai/«

gemini-3.8-flash -модель

Крок 2. Написати код

Напишемо коди (Рис.3.3).

providers.py > ...

```python
"""Єдиний інтерфейс для локальної Ollama та Google
import getpass
import os

PROVIDERS = {
    "local": {
        "base_url": "http://localhost:11434/v1",
        "model": "llama3.2:3b",
        "api_key": "ollama",
    },
    "cloud": {
        "base_url": "https://generativelanguage.goo
        "model": "gemini-3.5-flash-lite",
    },
}


def make_client(provider):
    if provider not in PROVIDERS:
        raise ValueError(f"Невідомий провайдер: {pr
    cfg = PROVIDERS[provider]

    if provider == "local":
        key = cfg["api_key"]
    else:
        key = os.getenv("CLOUD_API_KEY", "").strip()

        if not key:
            key = getpass.getpass(
                "Google API key (ввід прихований): "
            ).strip()
```

compare.py > ...

```python
import time

from providers import make_client

QUESTION = "Поясни різницю між стеком і чергою. Ко

for name in ["local", "cloud"]:
    client, model = make_client(name)

    t0 = time.perf_counter()
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": QUES
        temperature=0.7,
    )
    elapsed = time.perf_counter() - t0

    print(f"===== {name.upper()} ({model}) =====")
    print(response.choices[0].message.content)
    print(f"Час: {elapsed:.2f} с")
    print(response.usage)
    print()
```

Рис.3.3 –Створенні коди.

У подальших експериментах, після виникнення обмежень квоти API, хмарну модель було замінено на gemini-3.5-flash-lite. Початкові результати завдання 3 залишено для відображення першого етапу дослідження.

Крок 3. Запустити код

Запустимо коди (Рис.3.4).

```
(venv) deniza@Noutbuk-Deniza lab1 % python compare.py
===== LOCAL (llama3.2:3b) =====
Різниця між стеком та чергою полярної кристалічої структури:

- Стек: кристалічна структура, яка складається із індивідуальних молекул, з'єднаних між собою силовими зв'язками.
- Черга: кристалічна структура, яка складається із молекулярних ланцюжей, з'єднаних між собою stronger силовими зв
'язками, ніж у стеку.
Час: 6.47 с
CompletionUsage(completion_tokens=101, prompt_tokens=44, total_tokens=145, completion_tokens_details=None, prompt_
tokens_details=PromptTokensDetails(audio_tokens=None, cache_write_tokens=None, cached_tokens=43))

Traceback (most recent call last):
  File "/Users/deniza/lab1/compare.py", line 11, in <module>
    response = client.chat.completions.create(
  File "/Users/deniza/lab1/venv/lib/python3.9/site-packages/openai/_utils/_utils.py", line 298, in wrapper
    return func(*args, **kwargs)
  File "/Users/deniza/lab1/venv/lib/python3.9/site-packages/openai/resources/chat/completions/completions.py", lin
e 1284, in create
    return self._post(
  File "/Users/deniza/lab1/venv/lib/python3.9/site-packages/openai/_base_client.py", line 1360, in post
    return cast(ResponseT, self.request(cast_to, opts, stream=stream, stream_cls=stream_cls))
  File "/Users/deniza/lab1/venv/lib/python3.9/site-packages/openai/_base_client.py", line 1133, in request
    raise self._make_status_error_from_response(err.response) from None
openai.InternalServerError: Error code: 503 - [{'error': {'code': 503, 'message': 'This model is currently experie
ncing high demand. Spikes in demand are usually temporary. Please try again later.', 'status': 'UNAVAILABLE'}}]


(venv) deniza@Noutbuk-Deniza lab1 % python -c 'from dotenv import load_dotenv; import os; from openai import OpenA
I; load_dotenv(); c=OpenAI(api_key=os.getenv("CLOUD_API_KEY"), base_url="https://generativelanguage.googleapis.com
/v1beta/openai/"); r=c.chat.completions.create(model="gemini-3.8-flash", messages=[{"role":"user","content":"Поясн
и різницю між стеком і чергою. Коротко."}]); print(r.choices[0].message.content); print(r.usage)'
Головна різниця між ними — **у порядку доступу до елементів**:

* **Стек (Stack)** працює за принципом **LIFO** (*Last In, First Out* — «останній прийшов, перший пішов»).
  * **Приклад:** Стопка тарілок або чарок. Додати тарілку можна тільки зверху, і зняти можна тільки верхню.
  * **Доступ:** Тільки з одного боку (з вершини).

* **Черга (Queue)** працює за принципом **FIFO** (*First In, First Out* — «перший прийшов, перший пішов»).
  * **Приклад:** Звичайна черга в магазині. Хто перший прийшов до каси, той перший отримує товар і виходить.
  * **Доступ:** З обох боків (додаються нові в кінець, а виходять із початку).
CompletionUsage(completion_tokens=207, prompt_tokens=19, total_tokens=749, completion_tokens_details=None, prompt_
tokens_details=None)
(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.3.4 –Запуск кодів.

Крок 4. Заповнити таблицю (дані брати з response.usage).

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.8-flash
Час відповіді, с	6,47 с	2,57 с
completion_tokens	101	207
Довжина відповіді, символів	303	598
Мова відповіді	Українська	Українська

Фактичних помилок у відповіді	Є- модель неправильно пояснила стек і чергу	Немає

Крок 5. Відповісти на питання:

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: 0 рядків основного коду запиту. Завдяки сумісному API OpenAI для переходу достатньо змінити налаштування моделі: base_url, api_key і model у файлі providers.py.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна модель Gemini надала точнішу та зрозумілішу відповідь.

- Наприклад, вона правильно пояснила принципи LIFO для стека та FIFO для черги й навела зрозумілі приклади з повсякденного життя. Локальна модель неправильно зрозуміла запит і надала інформацію, не пов’язану з темою.

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так. Локальна модель допустила фактичні помилки, неправильно інтерпретувала запит і використала незрозумілі слова та невдалі мовні конструкції. Через це її відповідь була складною для розуміння та не відповідала завданню.

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: Хмарна модель використовує потужні сервери зі спеціалізованими прискорювачами для обробки запитів. Локальна модель працює на ресурсах власного комп’ютера, які можуть бути менш продуктивними. Тому обчислення на хмарних серверах можуть виконуватися швидше, навіть з урахуванням часу передавання даних через інтернет.

Крок 6. Питання до чат-бота:

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: Формат запитів OpenAI став стандартом, тому що він простий, зручний і широко підтримується. Інші провайдери, зокрема Ollama, використовують його для сумісності з наявними програмами без необхідності переписувати код.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 70 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Розмір моделі суттєво впливає на якість відповідей: модель на 70 млрд параметрів зазвичай краще розуміє складні запити та формує точніші відповіді, ніж модель на 3 млрд параметрів. Проте результат також залежить від якості навчальних даних, архітектури моделі та налаштувань. Тому більша модель не завжди означає кращу відповідь, але за інших подібних умов вона часто має перевагу.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Хмарна модель працює на потужних серверах зі спеціалізованими прискорювачами, тому може швидше обробляти запити. Це дозволяє компенсувати час передавання даних через інтернет. Локальна модель залежить від продуктивності комп'ютера.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Під час використання хмарних API існує ризик витоку персональних або конфіденційних даних, які передаються сервісу. У комерційних проєктах обмежують передавання чутливої інформації, шифрують дані, контролюють доступ і перевіряють політику зберігання даних провайдера.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit — це обмеження на кількість запитів або токенів за певний проміжок часу. Розробники працюють із ним за допомогою кешування відповідей, об'єднання запитів, повторних спроб із затримкою та використання кількох доступних провайдерів відповідно до їхніх правил. Також можна перейти на платний тариф із вищими лімітами.

Висновок за завданням 3

Під час виконання завдання було порівняно локальну модель Llama 3.2 (3B) та хмарну модель Gemini. Хмарна модель надала повнішу й точнішу відповідь, а також працювала швидше: 2,57 с проти 6,47 с у локальної моделі. Локальну модель доцільно використовувати, коли важливі конфіденційність даних, автономна робота та незалежність від інтернету, а хмарну — коли потрібні швидкість, точність і детальні відповіді.

Крок 5. Вимірювати швидкість для різних провайдерів

Виконання завдання:

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Затримка відповіді складається з двох різних величин, і одним числом її не описати:

- TTFT (time to first token) — час до появи першого слова. Поки він триває, користувач бачить порожній екран. Для локальної моделі це обробка вашого запиту, для хмарної — ще й дорога по мережі;
- Швидкість генерації — скільки токенів за секунду видається далі. Від неї залежить тривалість довгої відповіді.

  Ці показники можуть суперечити один одному: хмарна модель часто має гірший TTFT (пакет летить через пів світу), але кращу швидкість генерації. На короткій відповіді локальна виглядає швидшою, на довгій — програє.

  Щоб виміряти TTFT, відповідь треба отримувати по шматочках — це стрімінг, вмикається параметром stream=True. Той самий ефект, коли в чат-боті текст «друкується» на очах.

  Три правила коректного вимірювання: прогрів (перший запуск не рахуємо), три повтори з медіаною, без паралельних запитів.

  Варіанти (номер у журналі номер 22 тобто 2):

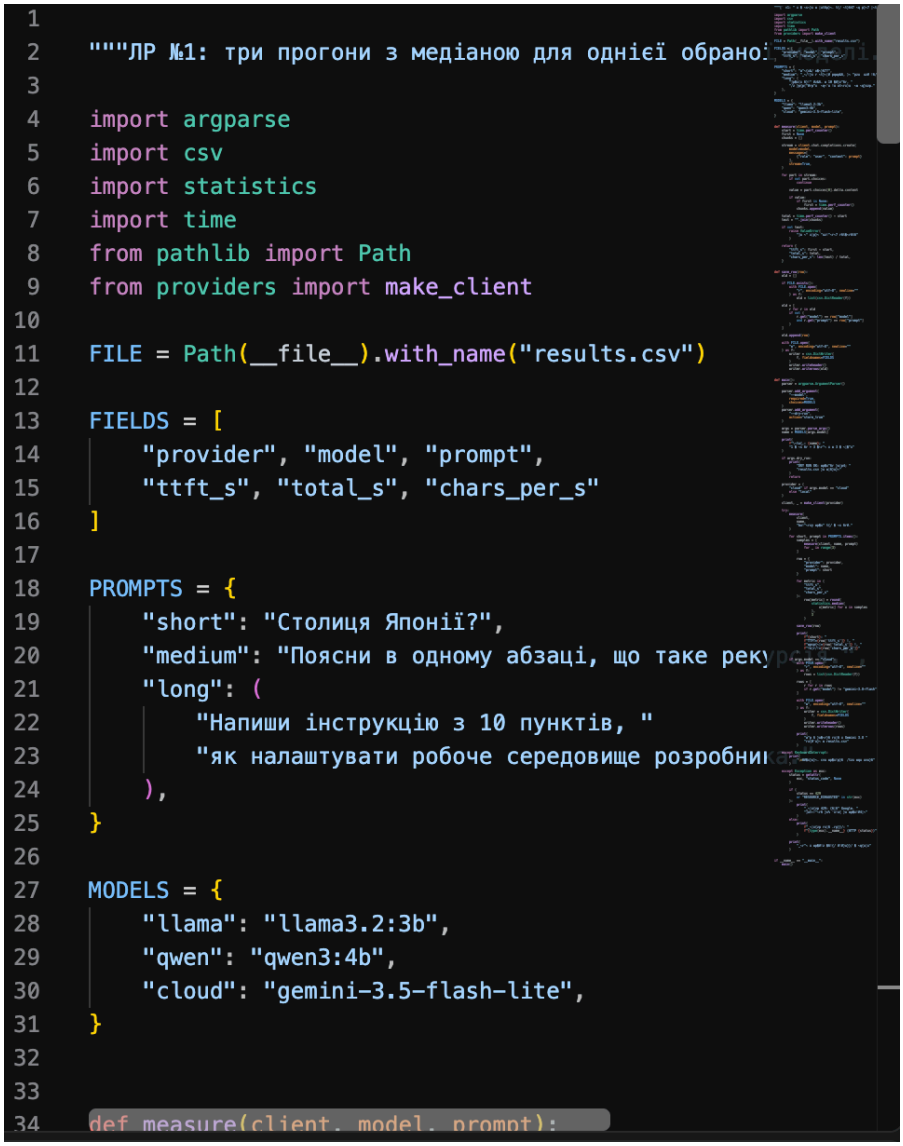
  Середній промпт для всіх однаковий: «Поясни в одному абзаці, що таке рекурсія.»

  Короткий промт — Столиця Японії?

  Довгий промпт — Напиши інструкцію з 10 пунктів, як налаштувати робоче середовище розробника.

Крок 1. Створити файл benchmark.py

Створимо код benchmark.py (Рис.4.1)



```python
"""ЛР №1: три прогони з медіаною для однієї обрано

import argparse
import csv
import statistics
import time
from pathlib import Path
from providers import make_client

FILE = Path(__file__).with_name("results.csv")

FIELDS = [
    "provider", "model", "prompt",
    "ttft_s", "total_s", "chars_per_s"
]

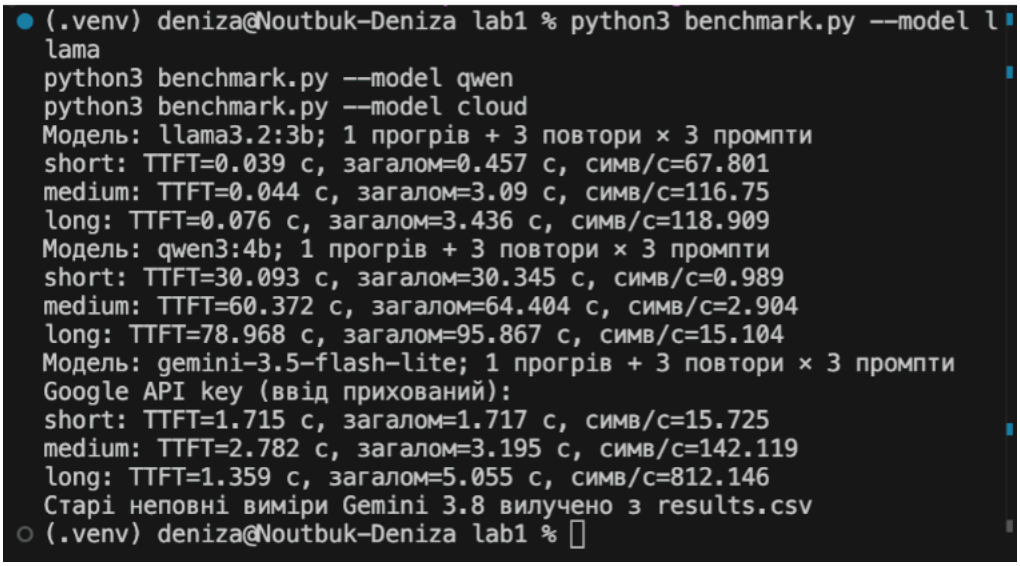
PROMPTS = {
    "short": "Столиця Японії?",
    "medium": "Поясни в одному абзаці, що таке реку
    "long": (
        "Напиши інструкцію з 10 пунктів, "
        "як налаштувати робоче середовище розробни
    ),
}

MODELS = {
    "llama": "llama3.2:3b",
    "qwen": "qwen3:4b",
    "cloud": "gemini-3.5-flash-lite",
}


def measure(client, model, prompt):
```

Рис.4.1 – Створення коду benchmark.py.

Крок 2. Запустити benchmark.py з параметром --model для кожної моделі. Результати вимірювань збережуться у файлі results.csv.
Запустимо код benchmark.py (Рис.4.2).



```
(.venv) deniza@Noutbuk-Deniza lab1 % python3 benchmark.py --model l
lama
python3 benchmark.py --model qwen
python3 benchmark.py --model cloud
Модель: llama3.2:3b; 1 прогрів + 3 повтори × 3 промпти
short: TTFT=0.039 с, загалом=0.457 с, симв/с=67.801
medium: TTFT=0.044 с, загалом=3.09 с, симв/с=116.75
long: TTFT=0.076 с, загалом=3.436 с, симв/с=118.909
Модель: qwen3:4b; 1 прогрів + 3 повтори × 3 промпти
short: TTFT=30.093 с, загалом=30.345 с, симв/с=0.989
medium: TTFT=60.372 с, загалом=64.404 с, симв/с=2.904
long: TTFT=78.968 с, загалом=95.867 с, симв/с=15.104
Модель: gemini-3.5-flash-lite; 1 прогрів + 3 повтори × 3 промпти
Google API key (ввід прихований):
short: TTFT=1.715 с, загалом=1.717 с, симв/с=15.725
medium: TTFT=2.782 с, загалом=3.195 с, симв/с=142.119
long: TTFT=1.359 с, загалом=5.055 с, симв/с=812.146
Старі неповні виміри Gemini 3.8 вилучено з results.csv
(.venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.4.2 – Запуск коду benchmark.py.

Крок 3. Окремо виміряти холодний і теплий старт локальної моделі: ollama stop llama3.2:3b

Одразу запустити benchmark.py і записати TTFT короткого промпту — це холодний старт. Запустити ще раз без зупинки моделі — це теплий.

Для виконання цього завдання було використано локальну модель llama3.2:3b. Спочатку виконано команду ollama stop llama3.2:3b, щоб зупинити модель перед вимірюванням холодного старту. Однак під час повторного запуску benchmark.py окремо виміряти холодний старт не вдалося, оскільки скрипт автоматично виконує попередній прогрів моделі перед основними вимірюваннями.

Тому для окремого вимірювання було підготовлено скрипт cold_warm.py (Рис.4.3), який надсилає короткий запит «Столиця Японії?» та вимірює час до отримання першого токена (TTFT). Перший запит використовується для вимірювання холодного старту, а повторний запит без зупинки моделі — для вимірювання теплого старту.

Такий підхід дає змогу порівняти час отримання першого токена під час першого запуску моделі та після її завантаження в пам’ять. Остаточні значення TTFT для холодного й теплого старту визначаються за результатами виконання cold_warm.py (Рис.4.4).

```
  UW PICO 5.09                    File: cold_warm.py                    Modified

import time
from providers import make_client

client, model = make_client("local")
prompt = "Столиця Японії?"

for name in ["Перший запит — холодний старт",
             "Другий запит — теплий старт"]:
    start = time.perf_counter()

    stream = client.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        stream=True,
    )

    first_token = None

    for chunk in stream:
        if not chunk.choices:
            continue
        if chunk.choices[0].delta.content:
            if first_token is None:
                first_token = time.perf_counter()

    if first_token is not None:
        print(f"{name}: TTFT = {first_token - start:.3f} c")
```

Рис.4.3 – Створення коду cold_warm.py.

Запуск коду cold_warm.py (Рис.4.4).

```
(venv) deniza@Noutbuk-Deniza lab1 % ollama stop llama3.2:3b
(venv) deniza@Noutbuk-Deniza lab1 % python3 cold_warm.py
Перший запит — холодний старт: TTFT = 8.045 с
Другий запит — теплий старт: TTFT = 0.072 с
(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.4.4 – Запуск коду cold_warm.py.

Крок 4. Заповнити таблиці:

Таблиця 6 – Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	Короткий	0,039	0,457	67,801
локальна	llama3.2:3b	Середній	0,044	3,090	116,750
локальна	llama3.2:3b	Довгий	0,076	3,436	118,909
хмарна	gemini-3.5-flash-lite	Короткий	1,715	1,717	15,725
хмарна	gemini-3.5-flash-lite	Середній	2,782	3,195	142,119
хмарна	gemini-3.5-flash-lite	Довгий	1,359	5,055	812,146

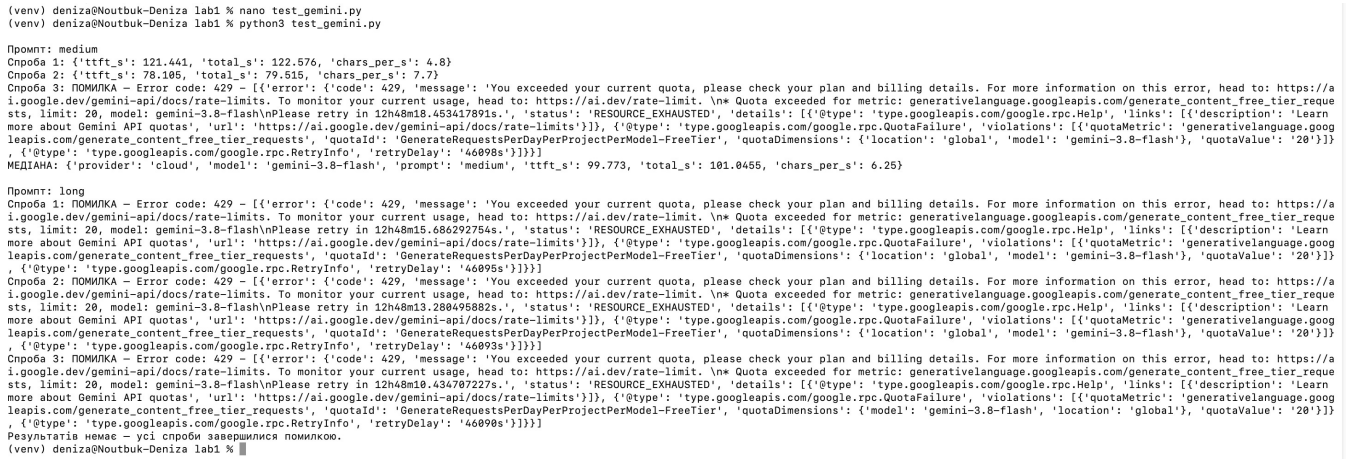


Рис.4.5 – Помилка API 429 RESOURCE_EXHAUSTED під час початкового вимірювання хмарної моделі.

Під час початкового вимірювання моделі gemini-3.8-flash виникли помилки 503 та 429 RESOURCE_EXHAUSTED, які не дозволили отримати повний набір результатів. Для повторного експерименту використано gemini-3.5-flash-lite. Було проведено вимірювання для трьох типів промптів із попереднім прогрівом і трьома повтореннями. Отримані значення медіани збережено у файлі results.csv та наведено в таблиці 6.

Таблиця 7 – Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	6,479	0,070
Всього, с	8,098	0,395

Зробимо тест для Таблиці 7 (Рис.4.6).

```
(venv) deniza@Noutbuk-Deniza lab1 % python3 cold_warm.py
Виконується холодний старт...
Холодний старт — TTFT: 6.479 с
Холодний старт — Всього: 8.098 с

Виконується теплий старт...
Теплий старт — TTFT: 0.07 с
Теплий старт — Всього: 0.395 с
(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.4.6 – Час холодного і теплого старту локальної моделі.

Крок 5. Побудувати стовпчикову діаграму (matplotlib): швидкість у символах за секунду для всіх шести комбінацій «модель × промпт». Можете запитати у LLM - як написати код для генерації стовпчикової діаграми.

Побудуємо стовпчикову діаграму швидкості генерації тексту для дев’яти комбінацій «модель × промпт»: трьох мовних моделей та трьох типів запитів. Діаграму створено на основі повторних вимірювань, збережених у файлі results.csv.

```python
import csv
import matplotlib.pyplot as plt

rows = []

with open("results.csv", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))

names = {
    "llama3.2:3b": "Llama 3.2",
    "qwen3:4b": "Qwen 3",
    "gemini-3.5-flash-lite": "Gemini Flash Lite",
}

prompts = {
    "short": "Короткий",
    "medium": "Середній",
    "long": "Довгий",
}

labels = [
    f"{names.get(r['model'], r['model'])}\n"
    f"{prompts.get(r['prompt'], r['prompt'])}"
    for r in rows
]

speeds = [float(r["chars_per_s"]) for r in rows]

plt.figure(figsize=(14, 7))
bars = plt.bar(range(len(rows)), speeds)

plt.xticks(
    range(len(rows)),
    labels,
    rotation=35,
    ha="right"
)

plt.ylabel("Швидкість, символів/с")
plt.title("Порівняння швидкості генерації мовних моделей")

for bar, value in zip(bars, speeds):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
```

Рис. 4.7 – Код побудови діаграми швидкості генерації на основі файлу results.csv.

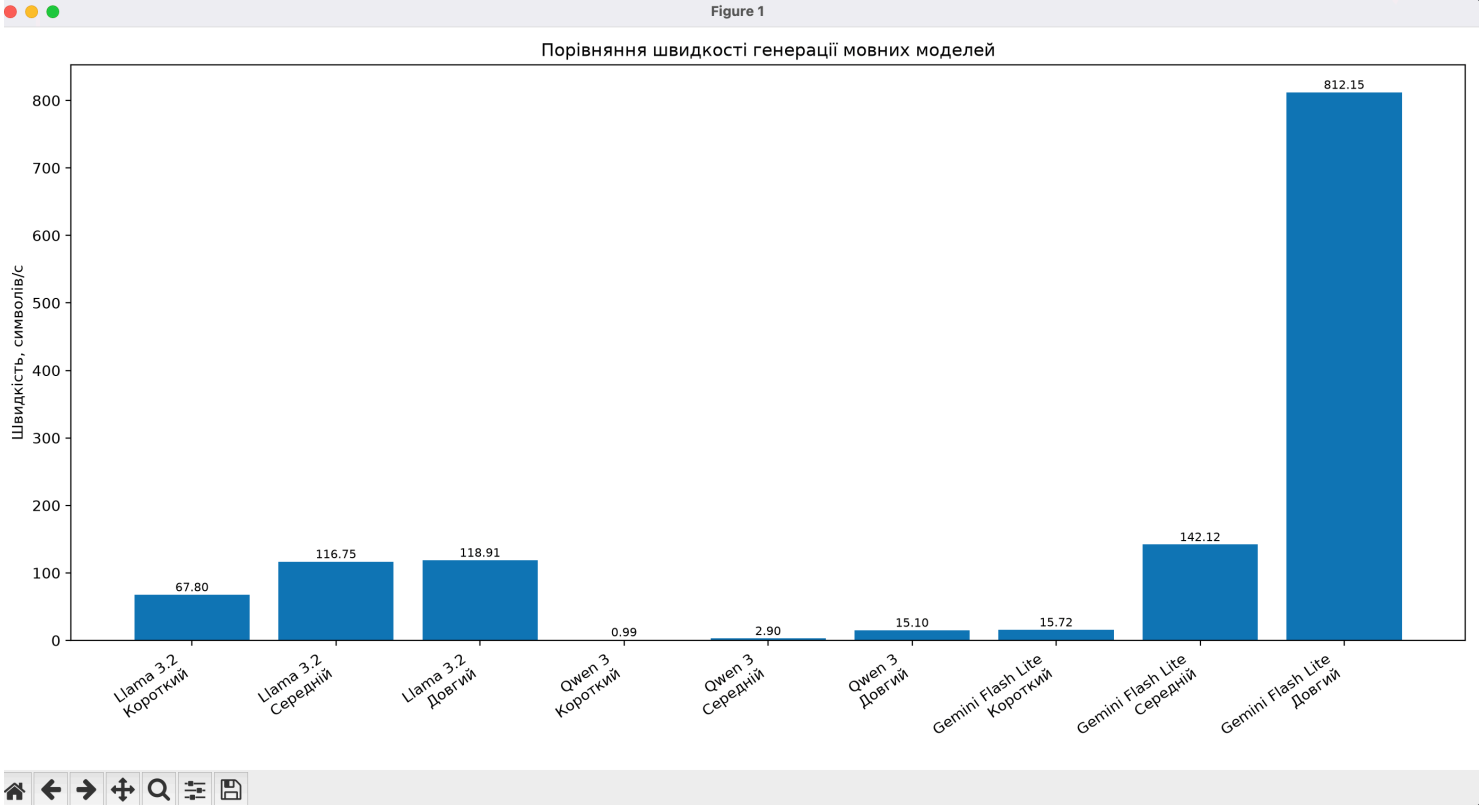


Рис. 4.8 – Порівняння швидкості генерації тексту трьох мовних моделей за результатами повторного експерименту.

Крок 6. Відповісти на питання (по 1–2 речення):

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: Найменший TTFT продемонструвала локальна модель llama3.2:3b — від 0,039 до 0,076 с. Це пояснюється тим, що модель працює локально та не потребує передавання запитів до зовнішнього сервера.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні —поясніть.

Відповідь: Ні, найменший TTFT не завжди означає найвищу швидкість генерації. Модель llama3.2:3b показала найменший час до першого видимого токена, тоді як gemini-3.5-flash-lite досягла найбільшої швидкості для довгого промпту — 812,146 символів/с.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: Для короткого промпту локальна модель llama3.2:3b працювала швидше за хмарну: 67,801 проти 15,725 символів/с. Для довгого промпту результати змінилися: gemini-3.5-flash-lite досягла 812,146 символів/с, тоді як llama3.2:3b — 118,909 символів/с.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Результати повторних вимірювань можуть відрізнятися через завантаження комп’ютера, час відповіді сервера та мережеві затримки. Три

повтори дають надійнішу оцінку, ніж один запуск, тому в експерименті використовувалася медіана результатів.

5. Чому холодний старт повільніший за теплий?

Відповідь: Під час холодного старту модель потрібно завантажити в пам’ять, що збільшує час до першого токена та загальний час відповіді. Під час теплого старту модель уже завантажена й готова до роботи, тому відповідає значно швидше.

Крок 7. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п’ять питань:

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT модель отримує та обробляє запит, виконує обчислення й генерує перший токен відповіді. Цей показник вимірюють окремо, оскільки він характеризує швидкість початку відповіді, тоді як загальна швидкість генерації показує, наскільки швидко модель створює весь текст.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Хмарна модель може мати більший TTFT через передавання запиту мережею та очікування обробки на сервері. Водночас потужне серверне обладнання може забезпечувати вищу швидкість генерації тексту після появи першого токена.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт — це запуск моделі, яка ще не завантажена в оперативну пам’ять. Під час першого запуску потрібно завантажити модель і підготувати її до роботи, тоді як під час наступних запусків вона вже готова генерувати відповідь.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Час виконання може змінюватися через навантаження системи, мережеві затримки та інші чинники. Медіана кількох запусків зменшує вплив випадкових відхилень і дає більш репрезентативний результат.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Під час обробки вхідного запиту модель може паралельно обчислювати представлення багатьох токенів. Генерація відповіді зазвичай відбувається послідовно, оскільки кожен наступний токен залежить від попередніх, тому створення тексту потребує більше часу.

У звіт додати дані таблиць, діаграму та стислі відповіді на питання (2–3 речення на кожну). Стисло напишіть свої думки: який із виміряних показників важливіший16 для інтерактивного чат-бота, а який — для фонової обробки великих обсягів тексту, і чому.

Висновок за завданням 4

На мою думку, для інтерактивного чат-бота найважливішим показником є TTFT, оскільки він визначає, наскільки швидко користувач побачить початок відповіді. Для фонової обробки великих обсягів тексту важливішою є швидкість генерації в символах за секунду, адже вона впливає на загальний час виконання завдання. Також варто враховувати загальний час відповіді, особливо під час обробки довгих запитів.

Підготовка середовища для виконання завдання 5:

Крок 6. Оцінити якість роботи моделей

У попередніх завданнях ми дізналися, яка модель швидша. Але швидкість сама по собі нічого не варта: модель, яка миттєво видає дурню, гірша за ту, що думає п’ять секунд і відповідає правильно.

Проблема в тому, що «якість» — поняття розпливчасте. Запис у звіті «модель А здалася розумнішою» — це не результат, а враження: його неможливо перевірити й неможливо відтворити.

Інженерне рішення — оцінювання за рубрикою: фіксований набір завдань, однаковий прогін через усі моделі, бали за заздалегідь визначеними правилами. Наприкінці курсу ви робитимете те саме для агентів.

Правило, яке не можна порушувати: набір завдань складається до того, як ви побачили відповіді. Підганяти завдання під результат — головний ворог чесного вимірювання.

Ще один нюанс: модель недетермінована, на повторний запуск того самого завдання вона може відповісти інакше. Тому кожне завдання прогоняється двічі. Розбіжність між прогонами — це не збій, а характеристика моделі, така сама важлива, як швидкість.

Використанні завдання

Кожне завдання ловить окремий тип поведінки моделі, і разом вони дають профіль її сильних і слабких місць. Нижче — що саме перевіряє кожне і на що дивитися при оцінюванні.

Завдання 1. Фактичні знання.

Що перевіряємо: чи знає модель факт і, головне, чи зізнається вона, що не знає.

Мовна модель не має бази даних, у якій можна подивитися відповідь. Вона генерує текст, який виглядає правдоподібно, спираючись на статистику мільярдів прочитаних текстів. Якщо потрібного факту в цій статистиці не було або він був представлений слабко, модель не мовчить — вона видає найправдоподібніший варіант. Це і називається галюцинацією: відповідь звучить упевнено й правильно оформлена, але вигадана.

Саме тому вам обов'язково треба знайти правильну відповідь в авторитетному джерелі, без цього ви не зможете відрізнити знання від впевненої вигадки, а сама модель вам не підкаже — вона однаково впевнена в обох випадках.

На що дивитися? Найпоказовіше — прогнати завдання двічі. Якщо модель обидва рази дала той самий рік, вона, найімовірніше, справді його знає. Якщо роки різні — вона їх вигадує, і навіть випадковий збіг із правильною відповіддю не рахується за знання.

Локальні моделі на 3–4 млрд параметрів особливо часто плавають на регіональних фактах: даних про українські установи в їхньому навчанні було мало.

Практичний висновок для курсу: саме через галюцинації існує RAG – підхід, коли моделі підсовують потрібні документи в контекст замість того, щоб покладатися на її пам'ять.

Завдання 2. Дотримання формату.

Що перевіряємо: чи можна використати модель як компонент програми, а не як співрозмовника?

Це найважливіше завдання набору з практичного погляду. Коли модель відповідає людині, невеликий безлад у форматі не заважає — людина зрозуміє. Але коли відповідь моделі йде далі в код, вона має бути машинозчитуваною. Один зайвий символ — і json.loads() падає з помилкою, а вся програма зупиняється.

Проблема в тому, що модель навчена на текстах, де код і JSON прийнято обгортати в markdown- блоки json . Ця звичка настільки міцна, що часто перемагає вашу пряму інструкцію «без markdown». Модель ніби «знає краще», як має виглядати правильно оформлена відповідь.

На що дивитися? Тут оцінка об’єктивна, суб’єктивності немає взагалі: пропустіть відповідь через json.loads() . Парситься одразу — 2 бали, треба зрізати обгортку — 1, не парситься — 0. Типові дефекти: markdown-обгортка, вступна фраза «Ось JSON:», кома після останнього поля, одинарні лапки замість подвійних.

Практичний висновок: саме тому в лабораторній 2 ви будете працювати не з «проханням дати JSON», а зі схемами Pydantic і повторними запитами при невдачі. Промпт — ненадійний спосіб керувати форматом, і ви маєте побачити це на власних числах, перш ніж вивчати надійні.

Завдання 3. Робота з наданим текстом.

Що перевіряємо: чи вміє модель працювати з інформацією, яку ви їй дали, не підмішуючи те, що вона «пам’ятає».

Це принципово інша задача, ніж перші дві. Там модель діставала відповідь із себе, тут — з тексту в запиті. Друге виходить у моделей значно надійніше, і на цьому побудована більшість реальних застосувань: підсумовування документів, витяг даних, відповіді на питання по базі знань.

Але й тут є вразливість — модель може непомітно спотворити дані. Замінити «25 тисяч» на «близько 20 тисяч», округлити відсоток, переплутати два роки місцями. У переказі це виглядає природно, і саме тому непомітно.

Навіщо потрібні числа в тексті? Це пастка на таке спотворення. Число або збігається з оригіналом, або ні — перевірка займає секунди й не потребує суб’єктивної оцінки. Без чисел довелося б на око вирішувати, «чи добре стиснуто», а це знову враження замість вимірювання.

Який текст шукати? Абзац на 5–6 речень за темою вашого варіанта, у якому є щонайменше три конкретні числа: роки, кількості, відсотки, суми. Джерело — Вікіпедія, офіційний сайт установи, новинна стаття. Для варіанта 1 (освіта) підійде абзац про університет із роком заснування, кількістю факультетів і кількістю студентів.

На що дивитися? 2 бали — два речення, усі числа на місці. 1 бал — числа правильні, але речень більше або додано те, чого в тексті не було. 0 балів — хоча б одне число спотворене чи втрачене.

Завдання 4. Генерація коду.

Що перевіряємо: чи придатний вивід моделі до безпосереднього запуску.

Код зручний для оцінювання тим, що він або працює, або ні — суб'єктивності мінімум. Але тут перевіряється не тільки правильність алгоритму, а й дві додаткові речі.

Перша — чи виконано умову повністю. У завданні є конкретна вимога: ігнорувати пробіли й регістр, відкинути найнижчу оцінку, порахувати з категорією. Модель може написати робочу функцію, яка цю вимогу не реалізує. Це типовий і небезпечний випадок: код запускається, тестів немає, помилку помічають пізно.

Друга — знову формат. Ви написали «лише код», а модель, найімовірніше, додасть пояснення до й після. Тут це не критично, але показує ту саму закономірність, що й у завданні 2: інструкції щодо формату виконуються ненадійно.

На що дивитися? Скопіюйте функцію, запустіть на 2–3 прикладах. 2 бали — працює й виконує всю умову. 1 бал — працює, але умову виконано частково, або навколо коду багато зайвого тексту. 0 балів — не запускається чи дає неправильний результат.

Завдання 5. Міркування в кілька кроків.

Що перевіряємо: чи здатна модель вивести відповідь, якої немає в готовому вигляді.

Це найважче завдання набору, і причина в самій механіці генерації. Модель видає текст по одному токену за раз. Коли ви просите одразу назвати число, усі проміжні обчислення мають вміститися в один крок — а це фізично обмежує складність задачі, яку модель витягне.

Задачі у варіантах спеціально влаштовані так, що перша інтуїтивна відповідь неправильна. У задачі про 25 студентів напрошується $25 - 12 - 9 = 4$, і модель часто саме це й пише — забувши, що четверо порахувалися двічі.

Ось чому в умові стоїть «покажи хід розв’язання». Коли модель розписує кроки, кожен проміжний результат стає текстом у контексті, на який спираються наступні кроки. Вона фактично використовує власний вивід як робочу пам’ять — і точність помітно зростає.

На що дивитися? Тут можливі два різні провали, і їх треба розрізняти. Перший: правильна відповідь без показаного ходу — інструкцію не виконано, 1 бал. Другий: розписаний хід, який виглядає переконливо, але містить арифметичну помилку — 0 балів. Другий випадок цінніший для звіту, бо демонструє, що правдоподібність міркування і його правильність — різні речі. Якщо все вірно - ставимо 2 бали.

Саме на цьому завданні найяскравіше видно різницю між міркувальною та інструктивною моделлю, яку ви вимірювали раніше.

Завдання 6. Якість української мови.

Що перевіряємо: те, що частіше за все ігнорують і потім наступають на це в реальному проєкті.

Мовні моделі вчилися переважно на англомовних текстах. Українських даних у навчанні на порядки менше, а російських — помітно більше, ніж українських. Наслідок передбачуваний: модель формально відповідає українською, але текст рясніє русизмами й кальками, іноді зривається на іншу мову посеред абзацу, а термінологію вживає неточно.

Це не косметична проблема. Якщо ви будуєте систему для українських користувачів, якість мови — така сама вимога, як швидкість чи точність. І що менша модель, то гірше вона з цим справляється.

Зверніть увагу на цікавий ефект, який ви, найімовірніше, побачите: модель може думати однією мовою, а відповідати іншою. Ви це вже спостерігали, коли міркувальна модель розписувала хід розв’язання російською чи англійською, а фінальну відповідь давала українською.

На що дивитися. 2 бали — природна українська, коректна термінологія, обидва абзаци по суті. 1 бал — суть правильна, але є русизми, кальки, кострубаті конструкції. 0 балів — відповідь не українською або перехід на іншу мову посеред тексту.

Як ці шість завдань працюють разом

Кожне окремо не дає повної картини — разом вони показують профіль моделі. Типовий результат, який ви побачите:

- мала локальна модель непогано справляється із завданнями 3 і 4 (робота з наданим текстом і простий код), провалює 1 і 5 (факти й міркування), плаває на 2 і 6 (формат і мова);
- хмарна велика модель упевнено бере 1, 3, 4, 6, але може так само проігнорувати вимогу «без markdown» у завданні 2 — і це важливе спостереження: розмір моделі не розв'язує проблему надійності формату.

Саме тому висновок «модель А краща за модель Б» майже завжди беззмістовний. Правильне формулювання — під яку задачу яка модель. І в наступних лабораторних, обираючи модель для агента, ви дивитиметесь передусім на завдання 2: агент викликає інструменти через структуровані дані, і модель, яка не тримає формат, зламає всю систему.

Виконання завдання:

Завдання 5. Оцінювання якості моделей за рубрикою

Шкала оцінювання:

Таблиця 8 – Загальна шкала оцінювання

Бал	Критерій
0	Не впорався: неправильна відповідь, зламаний формат, не та мова, відмова відповідати
1	Частково: суть правильна, але є дефект — зайвий текст навколо JSON, помилка в дрібниці, кострубата мова, відсутній хід розв'язання там, де його просили
2	Виконано повністю

Таблиця 9 – Критерії оцінювання за завданнями

№	2 бали	1 бал	0 балів
1	Відповідь збігається з авторитетним джерелом, обидва прогони однакові	Правильна відповідь, але прогони розійшлися	Відповідь не збігається з джерелом
2	json.loads() парсить без правок	Парситься після видалення markdown-обгортки або вступної фрази	Не парситься взагалі

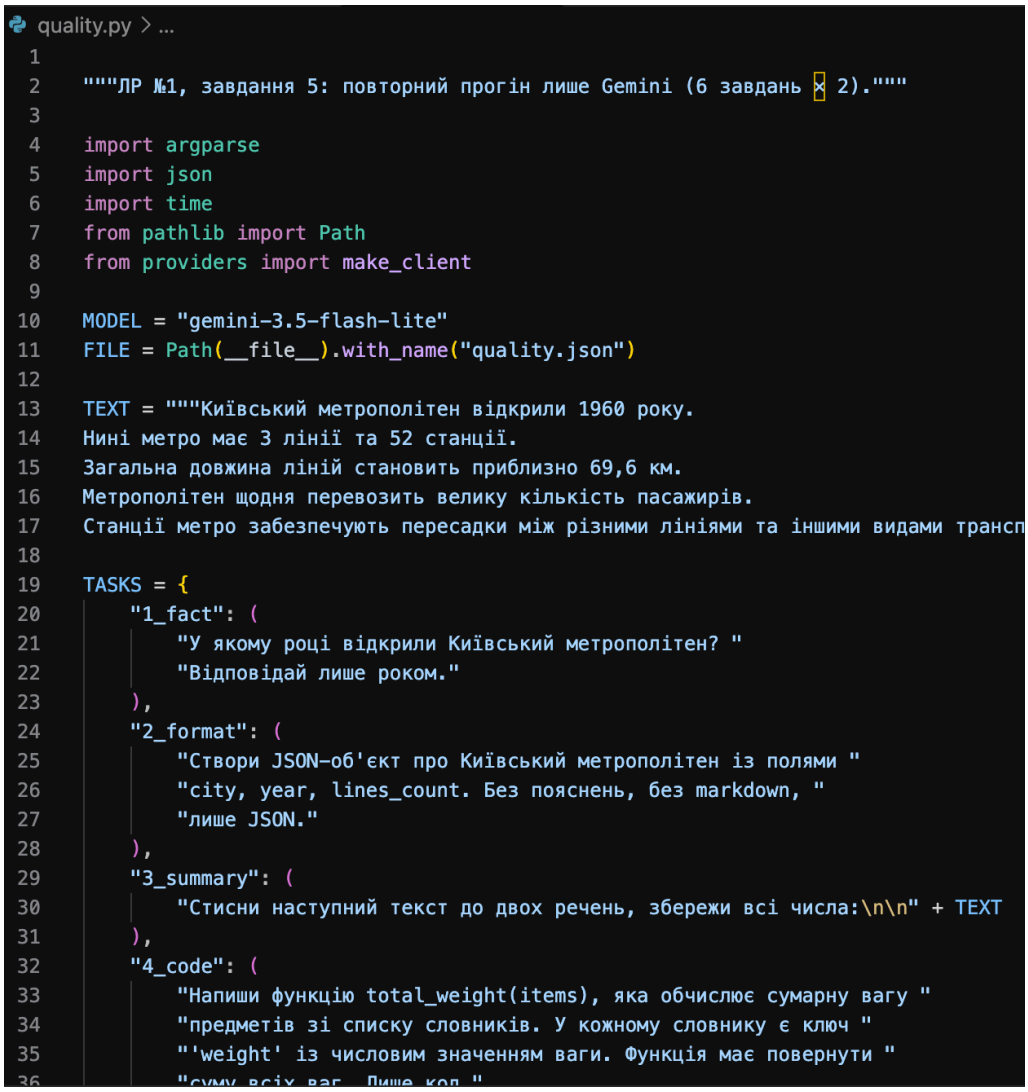
3	Два речення, усі числа збережені без спотворень	Числа правильні, але речень більше двох або додано інформацію, якої не було в тексті	Хоча б одне число спотворене або втрачене
4	Код запускається і виконує всю умову завдання	Код працює, але умову виконано частково, або навколо коду багато зайвого тексту	Не запускається або дає неправильний результат
5	Правильна відповідь і показаний хід без помилок	Правильна відповідь без ходу, або вірний підхід із дрібною помилкою в обчисленнях	Неправильна кінцева відповідь
6	Природна українська, коректна термінологія, обидва абзаци по суті	Суть правильна, але є русизми, кальки, кострубаті конструкції	Відповідь не українською або перехід на іншу мову посеред тексту

Маємо варіант номер 2 (за журналом 22). Тож виконаємо варіант: 2.

Завдання 1 (факт) - за темою Транспорт Рік відкриття Київського метрополітену
Завдання 2 (формат) - за темою Транспорт JSON: city, year, lines_count для Київського метро
Завдання 3 - студент знаходить сам, 5–6 речень за темою свого варіанта, обов’язково з числами. Формулювання: «Стисни до двох речень, збережи всі числа.»
Завдання 4 (код) - total_weight(items) — сума ваг зі списку словників
Завдання 5 (логіка) - 120 ящиків: 45 по 8 кг, решта по 12 кг. Вантажівка тримає 1200 кг. Скільки рейсів?
Завдання 6 (українська) - Чим контейнерне перевезення відрізняється від навалочного

Крок 1. Створити файл quality.py

На першому етапі було створено файл quality.py у середовищі Python (Рис.5.1). У файлі визначено шість завдань для оцінювання якості роботи мовних моделей: перевірка фактичних знань, дотримання формату JSON, робота з наданим текстом, генерація коду, логічне міркування та якість української мови. Для кожної моделі передбачено два прогони кожного завдання. Результати виконання зберігаються у файлі quality.json для подальшого аналізу та оцінювання за критеріями методичних вказівок.



```
quality.py > ...

"""ЛР №1, завдання 5: повторний прогін лише Gemini (6 завдань × 2)."""

import argparse
import json
import time
from pathlib import Path
from providers import make_client

MODEL = "gemini-3.5-flash-lite"
FILE = Path(__file__).with_name("quality.json")

TEXT = """Київський метрополітен відкрили 1960 року.
Нині метро має 3 лінії та 52 станції.
Загальна довжина ліній становить приблизно 69,6 км.
Метрополітен щодня перевозить велику кількість пасажирів.
Станції метро забезпечують пересадки між різними лініями та іншими видами трансп

TASKS = {
    "1_fact": (
        "У якому році відкрили Київський метрополітен? "
        "Відповідай лише роком."
    ),
    "2_format": (
        "Створи JSON-об'єкт про Київський метрополітен із полями "
        "city, year, lines_count. Без пояснень, без markdown, "
        "лише JSON."
    ),
    "3_summary": (
        "Стисни наступний текст до двох речень, збережи всі числа:\n\n" + TEXT
    ),
    "4_code": (
        "Напиши функцію total_weight(items), яка обчислює сумарну вагу "
        "предметів зі списку словників. У кожному словнику є ключ "
        "'weight' із числовим значенням ваги. Функція має повернути "
        "суму всіх ваг. Лише код."
```

Рис.5.1 – Створення файлу quality.py для оцінювання якості мовних моделей.

Крок 2. Запустити для всіх моделей: llama3.2:3b , qwen3:4b , хмарна. Відповіді збережуться у quality.json.

Під час початкового тестування моделі gemini-3.8-flash виникли помилки API 429 RESOURCE_EXHAUSTED, пов’язані з перевищенням доступної квоти. Для усунення цієї проблеми було обрано іншу хмарну модель — gemini-3.5-flash-lite. Повторний експеримент завершився успішно: отримано 12 відповідей на шість завдань у двох прогонах. Результати обох локальних моделей збережено без повторного запуску.

```
<-m pip install matplotlib\npython3 chart.py
Collecting matplotlib
  Using cached matplotlib-3.11.2-cp314-cp314-macosx_11_0_arm64.whl.metadata (80 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Using cached contourpy-1.4.0-cp314-cp314-macosx_11_0_arm64.whl.metadata (3.8 kB)
Collecting cycler>=0.10 (from matplotlib)
  Using cached cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.28.2 (from matplotlib)
  Using cached fonttools-4.68.0-cp314-cp314-macosx_10_15_universal2.whl.metadata (139 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Using cached kiwisolver-1.5.1-cp314-cp314-macosx_11_0_arm64.whl.metadata (5.2 kB)
Collecting numpy>=1.25 (from matplotlib)
  Using cached numpy-2.5.4-cp314-cp314-macosx_14_0_arm64.whl.metadata (6.6 kB)
Collecting packaging>=20.0 (from matplotlib)
  Using cached packaging-26.3-py3-none-any.whl.metadata (3.5 kB)
Collecting pillow>=9 (from matplotlib)
  Using cached pillow-13.0.0-cp314-cp314-macosx_11_0_arm64.whl.metadata (9.1 kB)
Collecting pyparsing>=3 (from matplotlib)
  Using cached pyparsing-3.3.3-py3-none-any.whl.metadata (5.9 kB)
Collecting python-dateutil>=2.7 (from matplotlib)
  Downloading python_dateutil-4.0.0-py3-none-any.whl.metadata (7.3 kB)
Collecting tzdata (from python-dateutil>=2.7->matplotlib)
  Using cached tzdata-2026.6-py2.py3-none-any.whl.metadata (1.4 kB)
Using cached matplotlib-3.11.2-cp314-cp314-macosx_11_0_arm64.whl (9.3 MB)
Using cached contourpy-1.4.0-cp314-cp314-macosx_11_0_arm64.whl (283 kB)
Using cached cycler-0.12.1-py3-none-any.whl (8.3 kB)
Using cached fonttools-4.68.0-cp314-cp314-macosx_10_15_universal2.whl (3.1 MB)
Using cached kiwisolver-1.5.1-cp314-cp314-macosx_11_0_arm64.whl (64 kB)
Using cached numpy-2.5.4-cp314-cp314-macosx_14_0_arm64.whl (5.5 MB)
Using cached packaging-26.3-py3-none-any.whl (129 kB)
Using cached pillow-13.0.0-cp314-cp314-macosx_11_0_arm64.whl (4.9 MB)
Using cached pyparsing-3.3.3-py3-none-any.whl (126 kB)
Downloading python_dateutil-4.0.0-py3-none-any.whl (92 kB)
Using cached tzdata-2026.6-py2.py3-none-any.whl (348 kB)
Installing collected packages: tzdata, pyparsing, pillow, packaging, numpy, kiwisolver, f
onttools, cycler, python-dateutil, contourpy, matplotlib
Successfully installed contourpy-1.4.0 cycler-0.12.1 fonttools-4.68.0 kiwisolver-1.5.1 ma
tplotlib-3.11.2 numpy-2.5.4 packaging-26.3 pillow-13.0.0 pyparsing-3.3.3 python-dateutil-
4.0.0 tzdata-2026.6

[notice] A new release of pip is available: 26.2.1 -> 26.3
[notice] To update, run: pip install --upgrade pip
<\nprint(f"Разом збережено: {total}/12")\nPY
Перевірка збережених відповідей: gemini-3.5-flash-lite
1_fact: 2/2 відповіді
2_format: 2/2 відповіді
3_summary: 2/2 відповіді
4_code: 2/2 відповіді
5_logic: 2/2 відповіді
6_language: 2/2 відповіді
Разом збережено: 12/12
(.venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.5.2 – Повторне тестування хмарної моделі gemini-3.5-flash-lite та збереження 12 відповідей у файлі quality.json.

Крок 3. Знайти правильну відповідь на завдання 1 в авторитетному джерелі. Без цього неможливо визначити, чи галюцинує модель.

Для перевірки правильності відповідей мовних моделей було використано авторитетне джерело — офіційний сайт Київського метрополітену - https://www.metropoliten.kiev.ua/hist.php?lang=1 . Згідно з історичною довідкою, Київський метрополітен розпочав роботу 6 листопада 1960 року.
Отже, правильна відповідь на завдання 1 — 1960 рік. Цей факт використовувався

як еталон для оцінювання відповідей моделей llama3.2:3b, qwen3:4b та gemini-3.5-flash-lite.

Крок 4. Перевірити завдання 2 через json.loads() для кожної моделі.

Для перевірки відповідей моделей було використано функцію json.loads() з бібліотеки json у Python. Вона дає змогу перевірити, чи відповідає отриманий текст формату JSON, і перетворити його на структуру даних Python.

У результаті повторного експерименту встановлено, що обидві відповіді моделі gemini-3.5-flash-lite на завдання 2 успішно проходять перевірку за допомогою json.loads(). Сформовані JSON-об'єкти містять поля city, year і lines_count з коректними значеннями: «Київ», 1960 та 3. Отже, модель дотрималася вимог щодо структури відповіді та правильності фактичних даних.

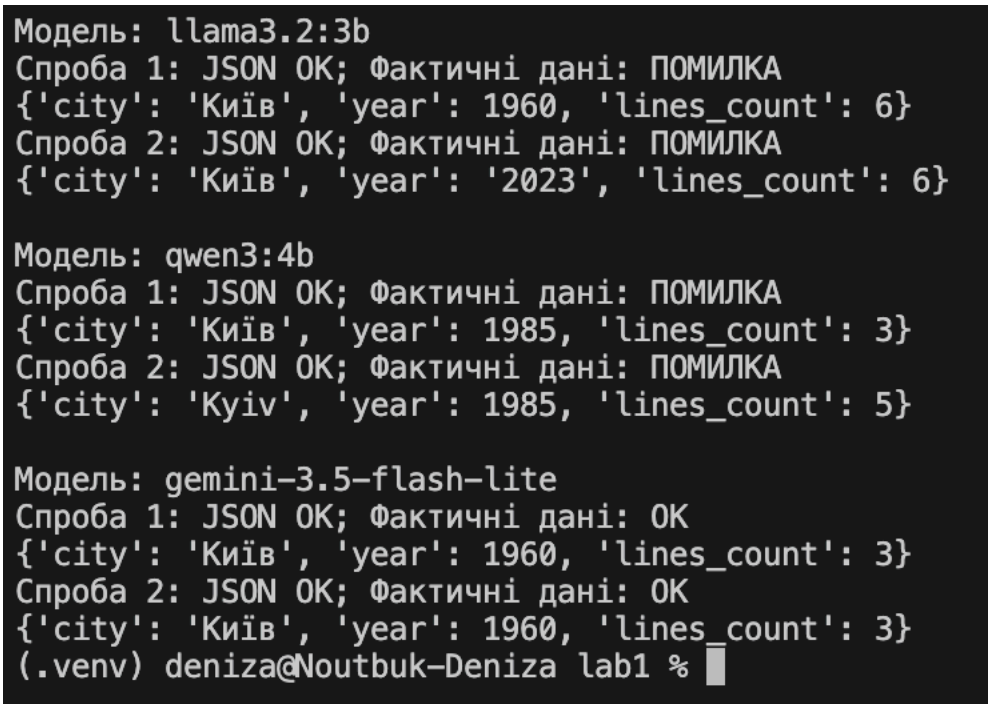


Рис.5.3 – Перевірка формату відповідей моделей за допомогою json.loads().

Крок 5. Відкрити quality.json , прочитати відповіді, виставити бали. Заповнити таблиці.

Таблиця 13 – Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	gemini-3.5-flash-lite
1. Факт	0	0	2
2. Формат	2	2	2
3. Стиснення	1	2	2
4. Код	2	2	1
5. Логіка	0	1	2
6. Українська	1	2	2
Разом /12	6	9	11

Після повторного експерименту з використанням хмарної моделі gemini-3.5-flash-lite було отримано 12 успішних відповідей на шість завдань. За результатами оцінювання модель набрала 11 із 12 балів. Єдиний недолік виявлено в завданні на генерацію коду: функція була правильною, проте модель додала Markdown-обрамлення, порушивши вимогу «лише код». Таким чином, повторне тестування дозволило отримати повноцінні результати для порівняння трьох моделей.

Таблиця 14 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	gemini-3.5-flash-lite
1. Факт	Ні	Ні	Ні
2. Формат	Ні	Ні	Ні
3. Стиснення	Так	Ні	Ні
4. Код	Ні	Ні	Ні
5. Логіка	Ні	Так	Ні
6. Українська	Ні	Ні	Ні

У двох прогонах хмарної моделі gemini-3.5-flash-lite оцінки за всіма шістьма завданнями збіглися. Це свідчить про стабільність результатів оцінювання в межах проведеного експерименту. Водночас тексти окремих відповідей відрізнялися, тому однакові бали не означають повної текстової відтворюваності.

Крок 6. Відповісти на питання (по 1–2 речення):

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільшу кількість балів отримала модель gemini-3.5-flash-lite — 11 із

12. Модель qwen3:4b набрала 9 балів, а llama3.2:3b — 6 балів. Основні відмінності спостерігалися в завданнях на фактичну точність, логічне міркування та якість української мови.
2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?
Відповідь: У llama3.2:3b оцінки розійшлися в завданні на стиснення, а в qwen3:4b — у завданні на логіку. Це свідчить, що результати моделей можуть змінюватися між запусками, тому надійність потрібно перевіряти кількома прогонами.
3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?
Відповідь: Так, усі три моделі — llama3.2:3b, qwen3:4b та gemini-3.5-flash-lite — повернули JSON без Markdown-обрамлення. Усі відповіді пройшли синтаксичну перевірку за допомогою json.loads(). Однак коректність формату не завжди означала правильність значень полів.
4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?
Відповідь: У відповідях локальних моделей траплялися русизми, кальки та неприродні мовні конструкції. Хмарна модель gemini-3.5-flash-lite продемонструвала кращу якість української мови, надавши змістовні та граматично коректні відповіді.

Крок 7. Обов'язковий пункт – приклад провалу. Обрати один найпоказовіший випадок і навести:

- повний текст запиту;
- повний текст відповіді, без скорочень;
- яка правильна відповідь мала бути;
- пояснення, чому так сталося.

1. Повний текст запиту

«У якому році відкрили Київський метрополітен? Відповідай лише роком.»

2. Повний текст відповіді

Перший запуск: `1928.`
Другий запуск: `1927.`

3. Яка правильна відповідь мала бути?

1960
Київський метрополітен відкрився 6 листопада 1960 року.

4. Пояснення, чому так сталося

Модель llama3.2:3b у двох прогонах згенерувала різні неправильні роки — 1928 та 1927 замість 1960. Це свідчить про фактичну помилку та нестабільність відповідей між запусками. Модель сформувала правдоподібні результати, але не перевірила їх за авторитетним джерелом.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свою підсумкову таблицю балів і поставити п’ять питань:

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: На практиці використовують стандартизовані бенчмарки, автоматичні метрики, наприклад BLEU та ROUGE, а також оцінювання відповідей експертами або іншими мовними моделями. Крім того, перевіряють швидкість роботи, використання ресурсів, стабільність відповідей і здатність моделі виконувати реальні завдання.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: Моделі часто навчені подавати код і структуровані дані в блоках Markdown, тому можуть автоматично додавати позначки на кшталт ```json навіть тоді, коли це заборонено. Щоб уникнути цього, використовують чіткі інструкції, перевірку результату парсером JSON або спеціальні режими структурованого виведення.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: У навчальних даних багатьох моделей англійська мова представлена значно ширше, ніж українська, тому моделі краще засвоюють її лексику, граматику та стилістичні особливості. Малі моделі також мають обмеженіші можливості для засвоєння мовних закономірностей, що може призводити до русизмів, кальок і неприродних формулювань.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Генерація тексту може бути недетермінованою: модель обирає наступні токени з певною ймовірністю, а параметри генерації, наприклад температура, впливають на варіативність результату. Тому навіть за однакового запиту відповіді можуть відрізнятися за точністю, повнотою та форматом.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація — це ситуація, коли модель генерує неправдиву або непідтверджену інформацію та подає її як достовірну. Це відбувається тому, що

модель навчається генерувати ймовірне продовження тексту, а не завжди перевіряти факти, і може не мати надійного механізму розпізнавання власного незнання.

Висновок за завданням 5

Під час виконання завдання було проведено порівняльне оцінювання трьох мовних моделей за шістьма критеріями. Локальна модель llama3.2:3b отримала 6 із 12 балів, qwen3:4b — 9 із 12, а хмарна gemini-3.5-flash-lite — 11 із 12.

Після заміни хмарної моделі вдалося отримати 12 успішних відповідей без помилок перевищення квоти API. Найвищу якість продемонструвала Gemini, проте в завданні на генерацію коду вона використала Markdown-обрамлення всупереч вимозі «лише код».

Отже, результати підтвердили необхідність комплексного оцінювання мовних моделей за фактичною точністю, правильністю формату, логічним міркуванням та якістю української мови.

Підготовка середовища для виконання завдання 6:

Крок 7. Дослідити вплив temperature і top_p

Модель не «пише текст». На кожному кроці вона рахує ймовірність для кожного можливого наступного токена. Наприклад, після фрази «Столиця України – » розподіл виглядає приблизно так:
- «Київ» → 0.94
- «місто» → 0.03
- «Одеса» → 0.01
- …ще тисячі варіантів з мізерними ймовірностями

Далі треба обрати один. За цей вибір і відповідають два параметри.

Temperature змінює контрастність розподілу перед вибором. При temperature = 0 завжди береться найімовірніший варіант — «Київ», щоразу. При підвищенні температури розподіл «сплющується»: різниця між імовірним і малоймовірним зменшується, і з’являється шанс обрати щось несподіване. При temperature = 1.5 модель цілком може написати «Одеса».

Top_p діє інакше — він обмежує коло кандидатів. При top_p = 0.1 модель бере лише найімовірніші варіанти, які разом набирають 10% усієї ймовірності, а решту викидає з розгляду.
При top_p = 1.0 розглядаються всі. Цей параметр не змінює співвідношення ймовірностей, він відсікає «хвіст».

Головне, що треба перевірити на досвіді: універсально правильного значення не існує. Для фактичних питань і видачі JSON потрібна низька температура — вам

потрібна передбачуваність. Для генерації ідей і назв потрібна висока — інакше модель щоразу видає ту саму нудну відповідь.

Виконання завдання:

Завдання 6. Керування випадковістю: temperature і top_p

Маємо варіант номер 2 (за журналом 22). Тож виконаємо варіант: 2.

Творче завдання: Назва застосунку нагадувань про ліки.
Фактичне питання: Рік відкриття Київського метрополітену.

Крок 1. Створити файл sampling.py :

Створення файлу sampling.py (Рис.6.1).

```
  UW PICO 5.09                                          File: sampling.py

from providers import make_client

client, model = make_client("local")

CREATIVE = (
    "Придумай одну оригінальну назву для застосунку "
    "нагадувань про приймання ліків. "
    "Назва має бути короткою, сучасною та легко запам'ятовуватися. "
    "Відповідай лише назвою."
)

FACTUAL = (
    "У якому році відкрився Київський метрополітен? "
    "Відповідай лише роком, без пояснень."
)


def run(prompt, n=5, **params):
    """Запускає промпт n разів і повертає список відповідей."""
    answers = []
    for _ in range(n):
        r = client.chat.completions.create(
            model=model,
            messages=[{"role": "user", "content": prompt}],
            **params,
        )
        answers.append(r.choices[0].message.content.strip())
    return answers
```

Рис.6.1 – Створення файлу sampling.py.

Крок 2. Творче завдання, вплив temperature:

У цьому кроці досліджується вплив параметра temperature на різноманітність відповідей локальної мовної моделі. Для творчого завдання використовується запит на створення назви застосунку нагадувань про приймання ліків — Pillora є прикладом такої назви, але модель генеруватиме власні варіанти.

Під час експерименту перевіряються три значення параметра: 0.0, 0.7 і 1.5.

Ключова метрика — кількість унікальних відповідей із п’яти. Це число йде в таблицю. Очікуваний результат: при 0.0 одна й та сама назва п’ять разів, при 1.5 — п’ять різних, але серед них можуть трапитись безглузді.

Додамо до файлу sampling.py фрагмент коду.

```
for t in [0.0, 0.7, 1.5]:
    answers = run(CREATIVE, temperature=t)
    print(f"temperature={t}: унікальних {len(set(answers))} з 5")
    for a in answers:
        print(" ", a)
    print()
```

Рис.6.2 – Доданий фрагмент до файлу sampling.py.

```
(venv) deniza@Noutbuk-Deniza lab1 % python3 sampling.py
temperature=0.0: унікальних 1 з 5
  MediMatch
  MediMatch
  MediMatch
  MediMatch
  MediMatch

temperature=0.7: унікальних 5 з 5
  MediBuddy
  Медкали
  MediPick
  "Дози"
  MediCheck

temperature=1.5: унікальних 5 з 5
  MINDDROP
  MedsHelper.
  Кадранія
  Firmax Alert
  Dozin

(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.6.3 – Виконання творчого завдання: вплив параметра temperature.

Крок 3. Творче завдання, вплив top_p.

Той самий цикл, але температура фіксована, змінюється top_p.

Порівняйте з попереднім кроком: обидва параметри впливають на різноманітність, але чи однаково? Дивіться на характер відповідей, не лише на їх кількість.

```
for p in [0.1, 0.5, 1.0]:
    answers = run(CREATIVE, temperature=0.7, top_p=p)
    print(f"top_p={p}: унікальних {len(set(answers))} з 5")
    for a in answers:
        print(" ", a)
    print()
```

Рис.6.4 – Доданий фрагмент для завдання вплив параметра top_p.

```
(venv) deniza@Noutbuk-Deniza lab1 % python3 sampling.py
temperature=0.0: унікальних 1 з 5
  MediMatch
  MediMatch
  MediMatch
  MediMatch
  MediMatch

temperature=0.7: унікальних 5 з 5
  Медвін
  MediTake
  MediMatch
  MediPill
  MediSync

temperature=1.5: унікальних 5 з 5
  "ДопалайТактично".
  Медхуб
  Ресцом
  DokMed
  Дакалс.

top_p=0.1: унікальних 2 з 5
  MediMatch
  MediGuide
  MediGuide
  MediGuide
  MediGuide

top_p=0.5: унікальних 5 з 5
  Медваль
  MediAlert
  Медіта
  MediFix
  MediGuide

top_p=1.0: унікальних 5 з 5
  Допам
  MediMatch
  Домой
  Медвір
  Дозаповір

(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.6.5 – Виконання творчого завдання: вплив параметра top_p.

Порівняння впливу параметрів

За результатами експерименту, при temperature = 0.0 модель п’ять разів згенерувала однакову назву MediMatch, тоді як при значеннях 0.7 і 1.5 отримано по п’ять різних відповідей. Для параметра top_p при значенні 0.1 було отримано лише дві унікальні назви, оскільки відповідь MediGuide повторилася чотири рази. При top_p = 0.5 і top_p = 1.0 усі п’ять назв були різними.

Обидва параметри впливають на різноманітність, але працюють по-різному. Temperature змінює розподіл імовірностей наступних токенів, а top_p обмежує набір токенів, доступних для вибору. У цьому експерименті низьке значення top_p = 0.1 помітно зменшило різноманітність відповідей.

Крок 4. Фактичне питання.

На цьому кроці перевіримо, чи впливає температура на стабільність фактичних відповідей локальної мовної моделі. Для варіанта № 2 запит стосується року відкриття Київського метрополітену.

Модель потрібно запустити по 5 разів із temperature = 0.0 та temperature = 1.5, а потім порівняти отримані роки з правильною відповіддю.

При високій температурі ви з великою ймовірністю побачите різні роки. Модель вигадуватиме. Це і є демонстрація того, що температура — не «регулятор розумності», а регулятор випадковості, і на фактичних задачах вона працює проти вас.

Правильну відповідь знайдіть в авторитетному джерелі, посилання вкажіть у звіті.

```
for t in [0.0, 1.5]:
    answers = run(FACTUAL, temperature=t)
    print(f"temperature={t}:", answers)
    print()
```

Рис.6.6 – Доданий фрагмент для завдання фактичного питання.

Правильна відповідь і джерело

Правильна відповідь — 1960 рік. Київський метрополітен офіційно відкрився 6 листопада 1960 року. Це підтверджує офіційний сайт Київського метрополітену.

У моїй моделі локальною при temperature=0.0 усі п’ять відповідей однакові, але всі неправильні — модель відповіла 1927, хоча правильний рік відкриття

Київського метрополітену — 1960. При температурі 1.5 вже краще 1 правильний результат.

```
(venv) deniza@Noutbuk-Deniza lab1 % python3 sampling.py
temperature=0.0: унікальних 1 з 5
  MediMatch
  MediMatch
  MediMatch
  MediMatch
  MediMatch

temperature=0.7: унікальних 5 з 5
  MediPick
  MedDrop.
  MediCue
  Помпак
  MediMate

temperature=1.5: унікальних 5 з 5
  Виру
  Доза
  Medmix
  Меміта
  Таба!

top_p=0.1: унікальних 2 з 5
  MediGuide
  MediMatch
  MediGuide
  MediMatch
  MediMatch

top_p=0.5: унікальних 4 з 5
  MediGuide
  MediMatch
  MediFix
  MediAlert
  MediMatch

top_p=1.0: унікальних 5 з 5
  "Дозировка"
  MediMing
  Медспар
  MedMed
  MediDose

temperature=0.0: ['1927.', '1927.', '1927.', '1927.', '1927.']

temperature=1.5: ['1935.', '1927;', '1926.', '1927.', '1960.']

(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.6.7 – Виконання фактичного питання.

Крок 5. Перевірка відтворюваності.

Швидше за все збіжаться. Але гарантії немає – на це впливають особливості обчислень з плаваючою комою й паралелізація. Тримайте це в голові: у наступних лабораторних, порівнюючи дві версії агента, частина різниці може бути просто шумом, а не наслідком ваших покращень.

```
answers = run(FACTUAL, n=10, temperature=0.0)

print("Відповіді:", answers)
print("Кількість унікальних відповідей:", len(set(answers)))
print("Усі відповіді однакові:", len(set(answers)) == 1)
```

Рис.6.8 – Доданий фрагмент перевірки відтворюваності.

```
(venv) deniza@Noutbuk-Deniza lab1 % python3 sampling.py
temperature=0.0: унікальних 1 з 5
  MediMatch
  MediMatch
  MediMatch
  MediMatch
  MediMatch

temperature=0.7: унікальних 5 з 5
  MedMatch
  Дивайся
  MediAlert
  Терм "Віта"
  Медіан

temperature=1.5: унікальних 5 з 5
  Медіку
  Мінутка
  Віра
  Містопills.
  DosageMates

top_p=0.1: унікальних 2 з 5
  MediMatch
  MediMatch
  MediMatch
  MediGuide
  MediGuide

top_p=0.5: унікальних 3 з 5
  MediMatch
  MediAlert
  MediAlert
  MediFix
  MediMatch

top_p=1.0: унікальних 5 з 5
  Медикалайн.
  MediKol.
  MedDrop
  MediPal
  Mediko

temperature=0.0: ['1927.', '1927.', '1927.', '1927.', '1927.']

temperature=1.5: ['1927', '1936', '1928.', '1928', '1966.']

Відповіді: ['1927.', '1927.', '1927.', '1927.', '1927.', '1927.', '1927.', '1927.', '1927.', '1927.']
Кількість унікальних відповідей: 1
Усі відповіді однакові: True
(venv) deniza@Noutbuk-Deniza lab1 %
```

Рис.6.9 – Виконання перевірки відтворюваності.

У результаті модель десять разів надала однакову відповідь — 1927.. Кількість унікальних відповідей становила 1, а перевірка len(set(answers)) == 1 повернула значення True.

Крок 6. Заповнити таблиці.

Таблиця 15 – Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	0
temperature	0.7	5	—
temperature	1.5	5	1
top_p	0.1	2	—
top_p	0.5	4	—
top_p	1.0	5	—

Таблиця 16 – Перевірка відтворюваності відповідей при temperature=0.0

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	Так

Крок 7. Відповісти на питання (по 1–2 речення).

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: Для агента, який має повертати JSON, я оберу temperature=0.0, оскільки низька температура робить відповіді більш передбачуваними та зменшує випадковість під час генерації. Це допомагає дотримуватися заданої структури, хоча правильність і формат JSON усе одно потрібно перевіряти.

2. Яке — генератору варіантів заголовків?

Відповідь: Для генератора варіантів заголовків я оберу temperature=1.5, оскільки висока температура збільшує різноманітність відповідей і дає змогу отримувати більш незвичні ідеї. Проте деякі варіанти можуть бути нелогічними або не відповідати темі.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: Параметр temperature змінює розподіл імовірностей наступних токенів, роблячи вибір більш або менш випадковим. Параметр top_p обмежує набір токенів для вибору, залишаючи найімовірніші варіанти, сумарна ймовірність яких досягає заданого порога.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Ні, temperature=0 не гарантує абсолютної відтворюваності за будь-яких умов, оскільки на результат можуть впливати особливості обчислень і паралелізації. У моєму експерименті всі 10 запусків дали однакову відповідь — 1927, тому кількість унікальних відповідей становила 1, хоча сама відповідь була неправильною.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з’явитися.

Відповідь: Однією з найбезглуздіших відповідей була назва застосунку для нагадувань про приймання ліків — «Лісні дихання», яка не відповідає призначенню програми. Такий результат міг з’явитися через високу температуру, яка збільшує випадковість вибору токенів і може призводити до незвичних або нелогічних формулювань.

Крок 8. Надіслати будь-якому чат-боту (ChatGPT, Gemini, Claude тощо) свої дані з таблиць і поставити п’ять питань:

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Параметр temperature змінює розподіл імовірностей наступного токена: низьке значення робить його більш концентрованим на найімовірніших варіантах, а високе — більш рівномірним. Тому зі збільшенням температури зростає різноманітність відповідей, але також може збільшуватися кількість помилок.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: temperature змінює розподіл імовірностей токенів, роблячи вибір більш або менш випадковим. top_p обмежує набір кандидатів для вибору токенами, сумарна ймовірність яких досягає заданого порога.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Краще спочатку змінювати один параметр, залишаючи інший незмінним, щоб зрозуміти вплив кожного з них на результат. Одночасне налаштування temperature і top_p ускладнює визначення причини змін у відповідях.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Навіть при temperature=0 абсолютна відтворюваність не гарантується в усіх умовах через особливості обчислень із плаваючою комою та паралельної обробки. У моєму експерименті всі 10 відповідей були однаковими — 1927, тому кількість унікальних відповідей становила 1, хоча сама відповідь була неправильною.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для витягу структурованих даних, зокрема JSON, зазвичай використовують низьку температуру — 0.0–0.2, щоб отримувати більш передбачувані результати. Для генерації творчого тексту можна використовувати вищі значення, наприклад 0.7–1.5, щоб збільшити різноманітність відповідей.

Висновок за завданням 6

Під час виконання завдання було досліджено вплив параметрів temperature і top_p на різноманітність та відтворюваність відповідей мовної моделі. За результатами експерименту, при temperature=0.0 усі десять відповідей були однаковими, проте неправильними. Зі збільшенням температури відповіді ставали різноманітнішими, але іноді менш логічними.

У наступних лабораторних роботах для генерації структурованих даних, зокрема JSON, я обиратиму temperature=0.0 або temperature=0.1, а top_p залишатиму на значенні за замовчуванням, щоб не змінювати одночасно декілька параметрів. Налаштування за замовчуванням може виявитися непридатним, оскільки підвищена випадковість здатна призводити до нестабільних відповідей, помилок у форматі та відхилень від заданої структури. Тому параметри потрібно підбирати відповідно до завдання, а отримані структуровані дані додатково перевіряти.

Крок 8. Звести все разом і зробити висновок

Заповніть підсумкову таблицю.

Таблиця 17 – Підсумкове порівняння моделей

Критерій	**llama3.2:3b (3B)**	**qwen3:4b (4B)**	**gemini-3.5-flash-lite**
Швидкість генерації, симв./с	67,801–118,909	0,989–15,104	15,725–812,146
TTFT, с	0,039–0,076	30,093–78,968	1,359–2,782
Якість, балів /12	6/12	9/12	11/12
Стабільність формату (JSON)	2/2 коректні	2/2 коректні	2/2 коректні
Потрібні ресурси	Ресурси локального ПК	Вищі обчислювальні витрати	Інтернет та доступ до API
Вартість	Без оплати API; локальні ресурси	Без оплати API; локальні ресурси	Залежить від тарифу та квот
Куди передаються дані	Залишаються на ПК	Залишаються на ПК	На сервери Google
Працює без інтернету	Так	Так	Ні
Коли обирати	Для швидких простих запитів та автономної роботи	Для складніших локальних завдань за достатніх ресурсів	Для завдань, де важливі якість відповідей і швидкість обробки великих текстів

За результатами повторних експериментів було встановлено, що локальна модель llama3.2:3b забезпечує найменший час до першого видимого токена — від 0,039 до 0,076 с. Модель qwen3:4b продемонструвала значно більші затримки — від 30,093 до 78,968 с, що пов’язано, зокрема, з додатковими обчисленнями під час формування відповіді.

Хмарна модель gemini-3.5-flash-lite показала TTFT від 1,359 до 2,782 с та досягла найбільшої швидкості генерації для довгого промпту — 812,146 символів/с. За результатами оцінювання якості вона також отримала найвищий бал — 11 із 12.

Отже, для швидких нескладних запитів та автономної роботи доцільно використовувати llama3.2:3b. Якщо важлива складніша обробка інформації без передавання даних зовнішнім сервісам, можна розглядати qwen3:4b з урахуванням її високої затримки. Для завдань, де пріоритетом є якість відповідей і швидкість обробки великих текстів, доцільно використовувати хмарну модель за наявності доступу до API.

Загальні висновки

У ході лабораторної роботи було проведено порівняльний аналіз локальних моделей llama3.2:3b і qwen3:4b та хмарної моделі gemini-3.5-flash-lite за показниками продуктивності, якості відповідей, стабільності формату та вимог до ресурсів.

Найменший час до першого видимого токена продемонструвала модель llama3.2:3b. Модель qwen3:4b потребувала більше часу на формування відповіді, але показала вищу якість за результатами тестування. Хмарна модель gemini-3.5-flash-lite отримала найвищу оцінку якості — 11 із 12 балів, а також найбільшу швидкість генерації для довгого промпту.

Отже, для швидких нескладних запитів та автономної роботи доцільно використовувати llama3.2:3b. Для складніших локальних завдань можна розглядати qwen3:4b з урахуванням її високої затримки. Для завдань, де важливими є якість відповідей та продуктивність, доцільно використовувати хмарну модель за умови доступності API.

Репозиторій на GitHub:
lab1/
├── report.pdf звіт: конфігурація, усі таблиці, графік, висновки
├── providers.py налаштування провайдерів
├── benchmark.py заміри швидкості
├── quality.py прогін завдань на якість
├── sampling.py експерименти з temperature / top_p
├── results.csv сирі дані замірів
├── quality.json збережені відповіді моделей
├── .gitignore з рядком .env
└── (файл .env у репозиторій НЕ потрапляє)